Universidad del País Vasco
Euskal Herriko Unibertsitatea
BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

GRADO EN INGENIERÍA INFORMÁTICA DE GESTIÓN Y SISTEMAS DE INFORMACIÓN

TRABAJO DE FIN DE GRADO

SISTEMA DE CONSULTA DE LAS ACTAS DEL PLENO DEL AYUNTAMIENTO DE BILBAO MEDIANTE RAG SOBRE KNOWLEDGE GRAPH E ÍNDICE VECTORIAL



Alumno: Lopez Sanchez, Peio

Director: Egaña Aranguren, Mikel

Curso: 2026-2027

Fecha: Bilbao, 3 de noviembre de 2026

Resumen Laburpena Abstract

Resumen:

Este trabajo aborda la consulta en lenguaje natural de las actas del Pleno del Ayuntamiento de Bilbao, publicadas en formato PDF en el portal municipal. A partir de las 236 actas correspondientes al periodo 2007–2026 se han construido dos sistemas de recuperación de información que parten de los mismos documentos y difieren en la estructura de datos que emplean. El primero, un RAG vectorial, recupera los fragmentos de texto más afines a la consulta y está orientado a preguntas sobre el contenido de un debate, como la posición defendida por un grupo político ante una proposición determinada. El segundo, un sistema GraphRAG, traduce la consulta a SPARQL sobre un Knowledge Graph RDF/OWL y está concebido para preguntas que exigen recorrer el corpus completo, como el número de proposiciones presentadas sobre una materia o el grupo que más iniciativas ha registrado en un mandato. Ambos sistemas se integran en una misma interfaz conversacional e indican la página del acta de la que procede cada respuesta. La evaluación se ha realizado sobre un conjunto de preguntas cuya respuesta se ha verificado en las actas. Los resultados muestran que las dos arquitecturas no son intercambiables: el sistema GraphRAG resuelve las preguntas de recuento y de ordenación sobre el corpus completo, que quedan fuera del alcance del RAG vectorial, mientras que este último tiende a ser más honesto en sus respuestas.

Palabras clave: *GraphRAG, RAG vectorial, Knowledge Graph, SPARQL, LLM, datos abiertos municipales.*

Laburpena:

Lan honek Bilboko Udalaren Osoko Bilkuraren aktak hizkuntza naturalean kontsultatzea du aztergai; akta horiek PDF formatuan argitaratzen dira udalaren atarian. 2007–2026 aldiari dagozkion 236 aktak oinarri hartuta, informazioa berreskuratzeko bi sistema eraiki dira, dokumentu beretatik abiatzen direnak eta erabiltzen duten datu-egituran bereizten direnak. Lehenak, RAG bektorial batek, kontsultarekin lotura handiena duten testu-zatiak berreskuratzen ditu, eta eztabaida baten edukiari buruzko galderetara bideratuta dago, hala nola talde politiko batek proposamen jakin baten aurrean defendatutako jarrera. Bigarrenak, GraphRAG sistema batek, kontsulta SPARQL lengoaiara itzultzen du RDF/OWL ezagutza-grafo baten gainean, eta corpus osoa zeharkatzea eskatzen duten galderetarako pentsatuta dago, hala nola gai bati buruz aurkeztutako proposamen kopurua edo agintaldi batean ekimen gehien erregistratu dituen taldea. Bi sistemak elkarrizketa-interfaze bakarrean integratzen dira, eta erantzun bakoitza aktaren zein orrialdetatik datorren adierazten dute. Ebaluazioa erantzuna aktetan egiaztatu zaien galdera multzo baten gainean egin da. Emaitzek erakusten dute bi arkitekturak ez direla elkarren ordezko: GraphRAG sistemak corpus osoaren gaineko zenbaketa- eta ordenatze-galderak ebazten ditu, RAG bektorialaren irismenetik kanpo geratzen direnak, eta azken horrek, berriz, erantzunetan zintzoagoa izateko joera du.

Gako-hitzak: *GraphRAG, RAG bektoriala, ezagutza-grafoa, SPARQL, LLM, udal datu irekiak.*

Abstract:

This work addresses natural-language querying of the minutes of the Plenary of Bilbao City Council, published in PDF format on the municipal website. From the 236 minutes corresponding to the period 2007–2026, two information retrieval systems have been built that start from the same documents and differ in the data structure they use. The first, a vector RAG, retrieves the text fragments most closely related to the query and is aimed at questions about the content of a debate, such as the position defended by a political group on a given proposal. The second, a GraphRAG system, translates the query into SPARQL over an RDF/OWL Knowledge Graph and is designed for questions that require traversing the entire corpus, such as the number of proposals submitted on a given subject or the group that registered the most initiatives during a term of office. Both systems are integrated into a single conversational interface and indicate the page of the minutes from which each answer is drawn. The evaluation was carried out on a set of questions whose answers have been verified against the minutes. The results show that the two architectures are not interchangeable: the GraphRAG system answers counting and ranking questions over the entire corpus, which lie beyond the reach of the vector RAG, whereas the latter tends to be more honest in its answers.

Keywords: *GraphRAG, vector RAG, Knowledge Graph, SPARQL, LLM, open municipal data.*

Índice

Resumen Laburpena Abstract **1**

Lista de figuras **6**

Lista de tablas **7**

Lista de acrónimos **9**

Empleo de Inteligencia Artificial (IA) **10**

1. Introducción **11**

1.1. Motivación 11

1.2. Objetivos del trabajo 12

1.3. Conexión con los Objetivos de Desarrollo Sostenible 12

1.4. Estructura de la memoria 13

2. Contexto **14**

2.1. Transparencia y digitalización de la administración pública 14

2.2. Large Language Models 14

2.3. Retrieval-Augmented Generation 15

2.4. Knowledge Graphs 15

2.5. GraphRAG 16

2.6. El Pleno del Ayuntamiento de Bilbao 16

3. Descripción de requerimientos y análisis del estado del arte **17**

3.1. Requisitos del sistema 17

3.2. Estado del arte de las tecnologías utilizadas 19

4. Planificación del proyecto **21**

4.1. Diagrama EDT 21

4.2. Descripción de tareas 21
4.3. Gestión del tiempo 27
4.4. Metodología de desarrollo 29
4.5. Evaluación económica 29
4.6. Análisis de riesgos 31

5. Análisis de alternativas **34**

5.1. LLM: ejecución local frente a API en la nube 34
5.2. Almacenamiento del grafo: librería en memoria frente a base de grafos dedicada 35
5.3. Recuperación vectorial: búsqueda híbrida frente a búsqueda semántica . . . 36
5.4. Modelo y *prompt* para la generación de SPARQL: estudio comparativo . . . 37

6. Descripción de la solución propuesta **40**

6.1. Arquitectura general del sistema 40
6.2. Construcción del Knowledge Graph 40
6.3. Sistema de recuperación de información 43
6.4. Generación de respuestas 44
6.5. Interfaz web del sistema 46
6.6. Flujo completo del sistema 47
6.7. Configuración y decisiones de implementación 48

7. Corpus y preprocesado de datos **50**

7.1. El corpus de actas del Pleno 50
7.2. Esquema general del preprocesado 51
7.3. Etapas comunes 52
7.4. Preprocesado para el índice vectorial 53
7.5. Preprocesado para el grafo 53
7.6. Resultado del preprocesado 55

8. Análisis de resultados **56**

8.1. Objetivo de la evaluación 56
8.2. Metodología 56
8.3. Resultados del RAG vectorial 58
8.4. Resultados del GraphRAG 61

8.5. Comparación de los dos sistemas 61
8.6. Límites del RAG vectorial 66
8.7. Límites del Knowledge Graph 66
8.8. Valoración global 67
8.9. Limitaciones 68

9. Conclusiones y trabajo futuro **70**

9.1. Conclusiones del trabajo 70
9.2. Cumplimiento de objetivos 71
9.3. Conclusiones sobre las pruebas realizadas 72
9.4. Contribuciones científico-técnicas 72
9.5. Trabajo futuro 73

Bibliografía **75**

Lista de figuras

1. Icono del Objetivo 16: Paz, justicia e instituciones sólidas 13
2. Icono del Objetivo 11: Ciudades y comunidades sostenibles 13
3. Diagrama EDT del proyecto 21
4. Diagrama de Gantt del proyecto (semanas desde el 22 de enero de 2026) . . 29
5. Arquitectura general: dos estructuras de datos sobre los mismos documentos 40
6. Clases principales de la ontología y propiedades que las relacionan 41
7. Selección de los triples de una proposición del grafo 43
8. Respuesta del RAG vectorial, con el enlace a la página del acta bajo cada bloque 45
9. Respuesta del GraphRAG y consulta SPARQL ejecutada. La interfaz muestra la consulta en un panel lateral, que aquí se ha colocado debajo . . 46
10. Recorrido de una pregunta en cada uno de los dos sistemas 47
11. Comienzo de una proposición en el acta del 18 de junio de 2008, a la izquierda, y en la del 29 de enero de 2026, a la derecha 51
12. Esquema del preprocesado: etapas comunes y proceso específico de cada estructura de datos 51
13. Reparto de las 188 respuestas de cada sistema en las siete rondas con preguntas nuevas 62

Lista de tablas

1. Requisitos funcionales del sistema 18
2. Requisitos no funcionales del sistema 19
3. Paquete de trabajo: 1. Gestión y planificación 22
4. Paquete de trabajo: 1.1. Reuniones con el director 22
5. Paquete de trabajo: 1.2. Gestión del tiempo 22
6. Paquete de trabajo: 2. Investigación y estudio previo 23
7. Paquete de trabajo: 2.1. Estudio del estado del arte 23
8. Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas 23
9. Paquete de trabajo: 3. Desarrollo del sistema 24
10. Paquete de trabajo: 3.1. Extracción y preprocesado del corpus 24
11. Paquete de trabajo: 3.2. RAG vectorial 24
12. Paquete de trabajo: 3.3. GraphRAG 25
13. Paquete de trabajo: 3.4. Interfaz web 25
14. Paquete de trabajo: 4. Evaluación del sistema 25
15. Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia . . 26
16. Paquete de trabajo: 4.2. Evaluaciones y comparativa 26
17. Paquete de trabajo: 4.3. Validación contra PDF real y regresión 26
18. Paquete de trabajo: 5. Documentación 27
19. Paquete de trabajo: 5.1. Redacción de la memoria 27
20. Paquete de trabajo: 5.2. Documentación técnica 27
21. Horas estimadas frente a dedicación real aproximada 28
22. Coste de mano de obra 29
23. Coste de hardware (amortizado) 30
24. Coste de software 30
25. Costes indirectos (electricidad) 31
26. Coste total del proyecto 31
27. Riesgo: 1. Dependencia de APIs externas 31

28. Riesgo: 2. Errores silenciosos en la generación de SPARQL 32
29. Riesgo: 3. Calidad variable del texto extraído de las actas 32
30. Riesgo: 4. Plazo de entrega ajustado con un único desarrollador 33
31. Riesgo: 5. Nombres y grupos ambiguos o mal atribuidos en las actas 33
32. Consultas correctas según la forma del *prompt* 37
33. Consultas correctas con la mejor forma de *prompt* de cada modelo; n es el número de consultas generadas 38
34. El corpus de actas del Pleno 50
35. Pasos del preprocesado y estructura de datos a la que sirve cada uno 52
36. Datos obtenidos del corpus 55
37. Evaluaciones realizadas 56
38. Métricas RAGAS del RAG vectorial en su configuración final (generador Gemini 3.5 Flash-Lite) 59
39. Diagnóstico de la recuperación: preguntas cuyo pleno correcto llega al contexto del generador 59
40. Experimento controlado: mismo contexto recuperado, distinto generador . . 60
41. Resultados de las siete rondas, por tipo de pregunta. C: correcta; P: parcial; I: incorrecta 62
42. Última ronda (20 preguntas), por tipo de pregunta 63
43. Ejemplos de respuestas de la última ronda 63
44. Rondas con preguntas nuevas a lo largo del desarrollo. R7 es la de la Tabla 42. En R2–R6 el RAG vectorial funcionó sin reordenación (cuota de Cohere agotada sin que el sistema lo advirtiera) 65

Lista de acrónimos

API Application Programming Interface
BERT Bidirectional Encoder Representations from Transformers
ECTS European Credit Transfer and Accumulation System
EDT Estructura de Desglose del Trabajo
GPU Graphics Processing Unit
GraphRAG Graph Retrieval-Augmented Generation
IA Inteligencia Artificial
LLM Large Language Model
OWL Web Ontology Language
PDF Portable Document Format
RAG Retrieval-Augmented Generation
RAGAS Retrieval-Augmented Generation Assessment
RAM Random Access Memory
RDF Resource Description Framework
RF Requisito Funcional
RNF Requisito No Funcional
SHACL Shapes Constraint Language
SKOS Simple Knowledge Organization System
SPARQL SPARQL Protocol and RDF Query Language
SQL Structured Query Language
TFG Trabajo Fin de Grado
VRAM Video Random Access Memory

Empleo de Inteligencia Artificial (IA)

En este trabajo se ha utilizado un asistente de Inteligencia Artificial generativa como herramienta de apoyo en el desarrollo del software, en la evaluación del sistema y en la redacción de la memoria. Su uso ha tenido carácter auxiliar y no ha sustituido el trabajo de análisis, diseño y toma de decisiones técnicas, que corresponde al autor.

En el desarrollo, el asistente se ha empleado como ayuda en la implementación y en la depuración del código, así como en su revisión y limpieza. El desarrollo iterativo del proyecto, en el que el sistema se fue ajustando a partir de sucesivas baterías de preguntas, dejó en el código funciones en desuso o repetidas, y el asistente se utilizó para localizarlas y eliminarlas o reorganizarlas, comprobando en cada paso que el comportamiento del sistema no variaba. En la evaluación, se ha empleado para generar las preguntas de las baterías y analizar los registros de ejecución.

En la elaboración de la memoria, se ha utilizado para obtener propuestas de redacción a partir del contenido y de los resultados del propio trabajo. En concreto, se ha empleado para reformular frases, mejorar la claridad del texto y corregirlo.

El código y el texto producidos con esta ayuda se han revisado antes de su incorporación, y los resultados se han contrastado con las actas originales. La responsabilidad sobre el contenido del trabajo, su validez y la correcta citación de las fuentes corresponde íntegramente al autor.

1. Introducción

Este trabajo se enmarca en el ámbito de la consulta de documentación pública municipal mediante *Large Language Models* (LLM). El Ayuntamiento de Bilbao publica en su portal web, para cada sesión del Pleno celebrada desde 2007, el orden del día, los extractos, un resumen y el acta completa. De esos documentos se toma el acta, por ser el único que recoge de forma íntegra las proposiciones, el debate y el resultado de cada votación. Sobre las actas publicadas entre 2007 y 2026 se desarrollan y se comparan dos sistemas de consulta en lenguaje natural: un RAG vectorial y un sistema GraphRAG.

Este capítulo presenta la motivación que ha llevado al desarrollo del proyecto, los objetivos planteados, su relación con los Objetivos de Desarrollo Sostenible y la estructura del resto de la memoria.

1.1. Motivación

Las actas del Pleno del Ayuntamiento de Bilbao son documentos públicos, pero consultarlas es costoso. Cada acta puede superar el centenar de páginas, y en ella las proposiciones, su debate y sus votaciones aparecen mezclados con abundante contenido de trámite administrativo. Separar lo uno de lo otro forma parte del problema que aborda este trabajo.

Por ello, localizar un dato concreto exige leer numerosos documentos o conocer de antemano el acta en la que se trató. Es el caso de preguntas habituales sobre la actividad municipal, como el sentido del voto de un grupo político en un asunto, el número de proposiciones presentadas sobre vivienda en un año o la evolución de un tema a lo largo de los mandatos. El portal municipal no ofrece ningún medio para formular estas preguntas en lenguaje natural y obtener una respuesta verificable en el documento original.

Los LLM actuales permiten plantear preguntas en lenguaje natural sobre un conjunto de documentos propio, mediante *Retrieval-Augmented Generation* (RAG), en la que el modelo responde a partir de los pasajes que el sistema localiza en esos documentos. Este trabajo aplica dos variantes de esa técnica al corpus de actas y las compara.

1.1.1. A quién beneficia el sistema

El sistema desarrollado está pensado para distintos perfiles. La ciudadanía puede conocer el resultado de una propuesta concreta sin saber en qué acta figura. Los periodistas pueden comparar la actividad de los grupos políticos, por ejemplo mediante el número de iniciativas presentadas por cada uno sobre una materia. El personal técnico municipal puede recuperar el precedente de un asunto ya tratado en sesiones anteriores. Por último, los investigadores disponen de un corpus estructurado para estudiar la actividad plenaria de casi dos décadas.

1.2. Objetivos del trabajo

1.2.1. Objetivo general y alcance

El objetivo general de este trabajo es desarrollar un sistema que permita consultar en lenguaje natural el contenido de las actas del Pleno del Ayuntamiento de Bilbao y que responda con datos verificables, enlazados a la página del documento de origen. Para ello se construyen dos sistemas que parten de los mismos documentos y los organizan en dos estructuras de datos distintas, un índice vectorial y un Knowledge Graph, y se comparan con el fin de determinar cuál de las dos es más adecuada para un corpus de este tipo. El trabajo abarca la totalidad de las actas publicadas entre 2007 y 2026.

1.2.2. Objetivos específicos

Del objetivo general se derivan los siguientes objetivos específicos:

1. Extraer de forma automatizada, a partir de las actas en PDF, las proposiciones de cada sesión con su fecha, el grupo que las presenta, el tema, las entidades mencionadas y el resultado de la votación (Capítulo 7).
2. Construir un Knowledge Graph RDF/OWL que modele las proposiciones, los grupos políticos, los concejales y los temas, y que admita razonamiento automático sobre la jerarquía de temas (Capítulo 6).
3. Implementar un sistema RAG vectorial con recuperación híbrida, que combine búsqueda semántica, literal, temática y por proposiciones sobre el mismo corpus.
4. Implementar un sistema GraphRAG que traduzca la pregunta del usuario a una consulta SPARQL sobre el grafo, orientado a las preguntas de agregación que el sistema vectorial no resuelve con precisión.
5. Integrar ambos sistemas en una interfaz web conversacional que indique, para cada dato, el documento y la página de origen.
6. Evaluar de forma comparativa ambos sistemas sobre un conjunto de preguntas cuya respuesta se ha verificado en las actas, con el fin de determinar en qué tipo de preguntas es más adecuada cada arquitectura (Capítulo 8).

1.3. Conexión con los Objetivos de Desarrollo Sostenible

En septiembre de 2015, los Estados miembros de las Naciones Unidas aprobaron la Agenda 2030, que fija 17 Objetivos de Desarrollo Sostenible orientados a reducir la desigualdad, proteger el medioambiente y garantizar el bienestar de las personas.

Este trabajo se relaciona principalmente con el Objetivo 16, *Paz, justicia e instituciones sólidas* (Figura 1), que persigue instituciones eficaces, responsables e inclusivas, y cuya meta 16.10 se refiere a garantizar el acceso público a la información. Las actas del Pleno son públicas, pero su extensión y su formato dificultan en la práctica su consulta. El sistema desarrollado facilita ese acceso al permitir preguntar en lenguaje natural y comprobar cada respuesta en el documento original, lo que favorece el seguimiento de la actividad municipal por parte de la ciudadanía, la prensa y la investigación.



Figura 1: Icono del Objetivo 16: Paz, justicia e instituciones sólidas
Fuente: Naciones Unidas.

De forma complementaria, el trabajo se relaciona con el Objetivo 11, *Ciudades y comunidades sostenibles* (Figura 2), cuya meta 11.3 promueve una urbanización inclusiva y una gestión participativa. Conocer qué se ha propuesto y qué se ha decidido en el Pleno es una condición previa para participar de forma informada en los asuntos municipales.



Figura 2: Icono del Objetivo 11: Ciudades y comunidades sostenibles
Fuente: Naciones Unidas.

1.4. Estructura de la memoria

La memoria se organiza del modo siguiente. El Capítulo 2 presenta los conceptos técnicos en los que se apoya el trabajo y el funcionamiento del Pleno del Ayuntamiento de Bilbao. El Capítulo 3 recoge los requisitos del sistema y el estado del arte. El Capítulo 4 expone la planificación temporal, la evaluación económica y el análisis de riesgos. El Capítulo 5 analiza las alternativas tecnológicas consideradas. El Capítulo 6 describe la solución desarrollada, formada por los dos sistemas de recuperación de información y la interfaz conversacional común. El Capítulo 7 describe el corpus de actas y su preprocesado. El Capítulo 8 presenta la evaluación de ambos sistemas y su comparación. Por último, el Capítulo 9 recoge las conclusiones y el trabajo futuro.

2. Contexto

Este capítulo sitúa el trabajo en su contexto. Expone el marco de transparencia administrativa en el que se publican las actas e introduce los conceptos técnicos en los que se apoya el sistema: los LLM, RAG, los Knowledge Graphs y GraphRAG. Por último, se describe el Pleno del Ayuntamiento de Bilbao, cuya documentación es el objeto del trabajo.

2.1. Transparencia y digitalización de la administración pública

La Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno [1], obliga a las administraciones públicas a publicar de forma activa la información relativa a su actividad. En paralelo, el movimiento de gobierno abierto defiende que esa información, además de publicarse, sea accesible y reutilizable.

La publicación de un documento no garantiza, sin embargo, su accesibilidad. Los portales municipales difunden las actas, los órdenes del día y los resúmenes de sesión en formato PDF, concebido para la lectura secuencial y poco adecuado para la consulta puntual o el análisis agregado. Cuando el volumen de documentos es elevado, como sucede con casi veinte años de actas del Pleno de Bilbao, la información está publicada, pero localizar un dato concreto exige una búsqueda manual que limita en la práctica quién puede utilizarla.

Las técnicas de Inteligencia Artificial que permiten consultar documentación extensa en lenguaje natural ofrecen un medio para reducir esa distancia entre publicación y acceso efectivo. Este trabajo las aplica a un caso concreto de documentación administrativa pública.

2.2. Large Language Models

Un LLM, siglas de *Large Language Model* o modelo de lenguaje de gran tamaño, es una red neuronal entrenada con grandes volúmenes de texto para predecir la continuación más probable de una secuencia. Tras un ajuste posterior orientado a seguir instrucciones, estos modelos resuelven tareas diversas a partir de una instrucción en lenguaje natural, el *prompt*, entre ellas responder preguntas, resumir documentos o mantener una conversación.

Su uso se generalizó a partir de la publicación de ChatGPT por OpenAI en noviembre de 2022 [2]. Desde entonces la oferta se ha ampliado con modelos propietarios y de código abierto, y hoy es posible elegir entre modelos accesibles mediante una API en la nube y modelos de menor tamaño ejecutables en un equipo local.

Un LLM presenta dos limitaciones relevantes para este trabajo. Por una parte, no dispone de información posterior a su entrenamiento ni de documentos específicos, como

las actas de un ayuntamiento. Por otra, puede generar afirmaciones verosímiles pero falsas, las llamadas alucinaciones [3]. En un dominio institucional, en el que cada dato debe poder comprobarse en el documento original, ambas limitaciones hacen necesario complementar el modelo con una fuente documental externa.

2.3. Retrieval-Augmented Generation

RAG, siglas de *Retrieval-Augmented Generation* o generación aumentada por recuperación [4], consiste en proporcionar al LLM, junto con la pregunta, información recuperada de una base de conocimiento externa, para que la respuesta se apoye en ella y no solo en lo aprendido durante el entrenamiento.

En su forma más habitual, un sistema RAG opera en dos fases. En la fase de preparación, los documentos se dividen en fragmentos (*chunks*), y cada fragmento se transforma en un vector numérico, o *embedding*, que representa su significado, de manera que los textos de contenido similar quedan próximos entre sí. Los vectores se almacenan en una base de datos vectorial. En la fase de consulta, la pregunta se transforma del mismo modo, se recuperan los fragmentos más próximos a ella y estos se incorporan al *prompt* a partir del cual el LLM genera la respuesta.

Este planteamiento permite responder sobre un corpus propio y actualizable sin reentrenar ningún modelo. Permite, además, indicar la fuente de cada afirmación, puesto que se conoce qué fragmentos, y por tanto qué documento y qué página, sustentan la respuesta.

El esquema descrito, sin más refinamientos, es el *Naive RAG*. Las variantes agrupadas bajo el nombre de *Advanced RAG* [5] añaden mejoras en alguna de sus fases, como la reformulación de la consulta o la reordenación de los fragmentos recuperados. Otras aproximaciones sustituyen el almacén de fragmentos por una base de conocimiento estructurada, como un Knowledge Graph, lo que da lugar a GraphRAG [6], que se describe en las Secciones 2.4 y 2.5.

En este trabajo se implementan dos sistemas sobre el mismo corpus: un RAG vectorial de tipo *Advanced RAG*, con recuperación híbrida y reordenación de resultados, y un sistema GraphRAG. Ambos se comparan sobre las mismas preguntas en el Capítulo 8.

2.4. Knowledge Graphs

Un Knowledge Graph, o grafo de conocimiento, representa la información como un conjunto de entidades y de relaciones entre ellas, en lugar de como texto libre [7]. El estándar empleado en este trabajo es RDF, *Resource Description Framework* [8], que expresa cada hecho como un triple formado por un sujeto, un predicado y un objeto. Así, el hecho de que una proposición ha sido presentada por un grupo político se representa con la proposición como sujeto, la relación de presentación como predicado y el grupo como objeto. Sobre RDF puede definirse una ontología en OWL, *Web Ontology Language* [9], que fija las clases y las relaciones válidas del dominio, y el grafo resultante se consulta con el lenguaje SPARQL [10].

Frente al texto libre, este modelo admite razonamiento automático, ya que un motor de inferencia deriva hechos que no figuran de forma explícita pero se siguen de la ontología. Por ejemplo, una proposición clasificada en el tema de los desahucios queda asociada también al tema de la vivienda, del que aquel depende, sin necesidad de registrar esa

relación en cada proposición. Esos hechos pueden deducirse al ejecutar cada consulta o calcularse de antemano y guardarse en el grafo, que es la opción adoptada en este trabajo, como describe la Sección 6.2.

2.5. GraphRAG

GraphRAG es la variante de RAG en la que la información se recupera de un Knowledge Graph y no de un almacén de fragmentos de texto. En el sistema de este trabajo, la pregunta se traduce a una consulta SPARQL, que el propio programa compone para los tipos de pregunta previstos y que genera un LLM en los demás casos. El motor de grafos ejecuta la consulta y devuelve datos exactos, y un LLM redacta la respuesta a partir de ellos.

Este planteamiento es adecuado para las preguntas de agregación, como la identificación del grupo que ha presentado más proposiciones sobre vivienda o el número de proposiciones aprobadas en cada año. Un sistema basado en fragmentos de texto no puede responderlas con precisión, ya que exigen contar y cruzar datos de todo el corpus y no localizar un pasaje concreto.

En esta memoria el término GraphRAG se emplea en sentido amplio, como RAG en el que lo recuperado procede de un Knowledge Graph. No designa el método que Edge et al. publicaron con ese nombre [6], en el que un LLM extrae del texto un grafo de entidades y relaciones, agrupa sus nodos en comunidades y redacta un resumen de cada una, y las preguntas globales se responden combinando esos resúmenes, sin lenguaje de consulta formal. El sistema de este trabajo parte de la misma limitación de la recuperación por fragmentos, pero la resuelve con una ontología definida manualmente y con consultas SPARQL. La relación entre ambos enfoques se retoma en la Sección 8.5.4 y en el trabajo futuro, en la Sección 9.5.

2.6. El Pleno del Ayuntamiento de Bilbao

Este trabajo se ha desarrollado sobre la documentación pública del Pleno del Ayuntamiento de Bilbao, órgano superior de gobierno y control del municipio, integrado por la alcaldía y los concejales elegidos en cada mandato. En sus sesiones, ordinarias o extraordinarias, los grupos políticos y la alcaldía presentan proposiciones que se debaten y se someten a votación. El resultado de cada votación, que puede ser la aprobación, la aprobación con enmienda o el rechazo de la proposición, o que esta decaiga, es decir, que quede sin efecto al aprobarse una enmienda que la sustituye, queda recogido en el acta de la sesión junto con el debate.

El Ayuntamiento publica las actas en su portal web, de acceso público, junto con el resto de la documentación de cada sesión. Ese archivo, de 236 actas de casi veinte años, forma el corpus de este trabajo. Su descripción y el proceso que lo transforma en datos estructurados se recogen en el Capítulo 7.

3. Descripción de requerimientos y análisis del estado del arte

Este capítulo recoge los requisitos que debe cumplir el sistema y revisa el estado del arte de las técnicas en las que se apoya. El corpus de actas sobre el que se aplican se describe en el Capítulo 7.

3.1. Requisitos del sistema

Los requisitos se derivan de los objetivos específicos fijados en la Sección 1.2.2 y se dividen en funcionales y no funcionales.

3.1.1. Requisitos funcionales

La Tabla 1 recoge los requisitos funcionales, que describen lo que el sistema debe hacer. Los tres primeros corresponden a la preparación de los datos, RF4 y RF5 a los dos sistemas de consulta que se comparan, RF6 y RF7 a la forma en la que ambos se ofrecen al usuario, y los dos últimos a la comprobación del trabajo.

ID	Descripción
RF1	**Extracción.** Extraer de forma automatizada, de cada acta en PDF, las proposiciones con su fecha, el grupo o particular que las presenta, el tema, las entidades mencionadas, el resultado de la votación y la página de origen.
RF2	**Knowledge Graph.** Construir un grafo RDF/OWL que modele proposiciones, grupos políticos, concejales y temas, con razonamiento automático sobre la jerarquía de temas definida en SKOS [11].
RF3	**Votos y resultados.** Derivar el resultado de la votación y el voto nominal por concejal mediante una capa determinista basada en el texto literal del acta, que prevalezca sobre la clasificación del LLM cuando el acta permite verificarla.
RF4	**RAG vectorial.** Implementar un sistema de RAG vectorial con recuperación híbrida, que combine búsqueda semántica, literal, temática y por proposiciones, y con reordenación de resultados.
RF5	**GraphRAG.** Implementar un sistema que traduzca la pregunta del usuario a una consulta SPARQL, la ejecute sobre el grafo y redacte la respuesta a partir de los datos obtenidos.
RF6	**Cita de la fuente.** Indicar, para cada dato mostrado por cualquiera de los dos sistemas, el documento y la página del PDF de origen, cuando el dato procede de un acta concreta.
RF7	**Interfaz.** Ofrecer ambos sistemas mediante una interfaz web conversacional común.
RF8	**Validación del grafo.** Validar la integridad estructural del grafo mediante restricciones SHACL [12], de forma independiente del código que lo construye.
RF9	**Evaluación.** Evaluar de forma comparativa ambos sistemas sobre un conjunto de preguntas de referencia cuya respuesta se ha verificado en las actas.

Tabla 1: Requisitos funcionales del sistema

3.1.2. Requisitos no funcionales

La Tabla 2 recoge los requisitos no funcionales, que fijan las condiciones en las que debe operar el sistema. Los dos primeros responden a la naturaleza institucional del corpus, en el que cada dato debe poder comprobarse en el acta, y los tres restantes, a las condiciones del proyecto.

ID	Descripción
RNF1	**Trazabilidad.** Toda respuesta debe poder contrastarse con el documento original; no se admite una respuesta sin fuente citable.
RNF2	**Precisión antes que cobertura.** Ante una pregunta que no pueda responder con datos verificables, el sistema debe indicarlo de forma explícita en lugar de generar una respuesta verosímil sin respaldo en el corpus.
RNF3	**Coste.** El sistema debe poder funcionar con LLM locales o con servicios en la nube gratuitos o de bajo coste, sin depender de un presupuesto de inferencia elevado.
RNF4	**Idioma.** Las consultas y las respuestas se formulan en castellano, aunque el corpus contiene texto en euskera.
RNF5	**Mantenibilidad.** La capa determinista y la basada en LLM deben mantenerse separadas, de forma que un error sistemático de clasificación pueda corregirse sin repetir el enriquecimiento con el LLM.

Tabla 2: Requisitos no funcionales del sistema

3.2. Estado del arte de las tecnologías utilizadas

Los conceptos generales de LLM, RAG y GraphRAG se han presentado en el Capítulo 2. Esta sección se centra en la reordenación de resultados en sistemas RAG y en la generación de consultas formales a partir de lenguaje natural, dos líneas de investigación más específicas en las que se apoya el trabajo. La generación de consultas formales es el componente más crítico del sistema GraphRAG, ya que una consulta errónea no produce un fallo visible, sino una cifra incorrecta que se presenta como un dato del grafo.

3.2.1. Recuperación híbrida y reordenación de resultados

La combinación de varios canales de recuperación con una reordenación posterior de los resultados es una de las mejoras que distinguen a *Advanced RAG* del esquema *Naive RAG* [5]. La reordenación se realiza habitualmente con un modelo *cross-encoder*, que evalúa de forma conjunta la pregunta y cada fragmento candidato, a diferencia de la búsqueda por *embeddings*, que los representa por separado. Esta técnica se propuso para la recuperación de pasajes con BERT [13] y se ha generalizado como medio para mejorar la precisión de los primeros resultados de un motor de recuperación semántica. En este trabajo se aplica a los candidatos obtenidos por los cuatro canales de búsqueda del RAG vectorial, que se describen en el Capítulo 6. Para ello se emplea el servicio de reordenación de Cohere [14], una empresa que ofrece LLM y modelos de reordenación a través de una API en la nube.

3.2.2. Fiabilidad de la generación de SPARQL con LLM

La traducción de lenguaje natural a consultas formales mediante LLM es un área de investigación activa, tanto para SPARQL sobre Knowledge Graphs, el caso del sistema GraphRAG de este trabajo, como para SQL sobre bases de datos relacionales, un problema de la misma estructura. A continuación se resume lo que recoge la literatura sobre los cuatro aspectos que han condicionado el diseño del sistema: el uso de ejemplos en el *prompt*, la manera de elegirlos, el tamaño del modelo y las plantillas de consulta.

En cuanto al uso de ejemplos, la generación sin ellos, o *zero-shot*, no basta para las consultas complejas. En una evaluación de varios modelos a los que no se proporciona ninguna consulta de referencia, el acierto es alto en las consultas sencillas, pero cae de forma notable en las complejas. Los autores señalan como vía de mejora la inclusión en el *prompt* de preguntas ya resueltas con su consulta, que sirven de ejemplo al modelo (*few-shot*) [15].

Lo habitual es buscar los ejemplos que más se parecen a cada pregunta, aunque no es la única opción. Un trabajo sobre un Knowledge Graph de accidentes de aviación emplea ejemplos fijos, escritos a mano, junto con información del grafo relacionada con la pregunta, y mejora en un 6 % el resultado del trabajo de partida sin necesitar un conjunto de preguntas resueltas del que elegirlos [16]. En *text-to-SQL*, la evaluación comparativa de Gao et al. muestra que elegirlos por su similitud con la pregunta y con la consulta supera a la elección al azar en los cuatro modelos probados [17]. Por ello, en este trabajo se ha comparado esa selección con el uso de los mismos ejemplos en todas las preguntas. La selección por similitud ha mejorado claramente a los modelos más capaces, y en los modelos pequeños su efecto depende de la forma del *prompt* (Capítulo 5).

Respecto al tamaño del modelo, un modelo mayor no garantiza mejores consultas, ya que la construcción del *prompt* influye tanto o más que el número de parámetros. Así, modelos de entre 1.000 y 15.000 millones de parámetros ajustados a la tarea pueden superar a modelos mucho mayores sin ese ajuste [18].

La alternativa a que el modelo escriba la consulta son las plantillas, consultas ya escritas en las que solo faltan los datos de la pregunta. En este enfoque (*template-based question answering*), propuesto en 2012 para la pregunta-respuesta sobre datos RDF [19], la tarea se reduce a dos pasos: seleccionar la plantilla de consulta que corresponde a la pregunta y rellenar sus huecos con las entidades y las relaciones que esta menciona [20]. Su ventaja es la fiabilidad, aunque solo dentro del catálogo de preguntas previsto, ya que esos sistemas tienden a sobreajustarse y generalizan con dificultad a formas de pregunta no vistas [20].

Trabajos recientes combinan las plantillas con la generación por LLM. Arazzi et al. almacenan las plantillas en una base de datos vectorial, recuperan por similitud las más próximas a la pregunta y se las proporcionan al LLM como contexto, junto con la ontología del grafo. En su evaluación, prescindir de las plantillas reduce al 79,2 % la proporción de consultas que se ejecutan sin error y empeora la exactitud de los resultados [21]. El sistema GraphRAG de este trabajo adopta también un planteamiento híbrido, aunque dispuesto de otro modo: las preguntas de las formas más frecuentes se resuelven con consultas compuestas por el propio programa o con plantillas seleccionadas por similitud de *embeddings*, sin intervención de un LLM, y solo las restantes se delegan en la generación por LLM.

Estos resultados se contrastan en el Capítulo 5 con los obtenidos sobre el grafo de este trabajo y motivan dos decisiones de diseño que se describen en el Capítulo 6: recurrir al LLM para escribir la consulta solo cuando no hay otra vía y someter las consultas que genera a una capa determinista de corrección.

4. Planificación del proyecto

Este capítulo presenta la planificación del proyecto: la estructura de desglose del trabajo, la descripción de los paquetes que la componen, la gestión del tiempo, la metodología de desarrollo, la evaluación económica y el análisis de riesgos.

4.1. Diagrama EDT

La Figura 3 muestra la Estructura de Desglose del Trabajo (EDT) del proyecto, con los cinco paquetes de trabajo de primer nivel y sus subpaquetes, que se detallan en la Sección 4.2.

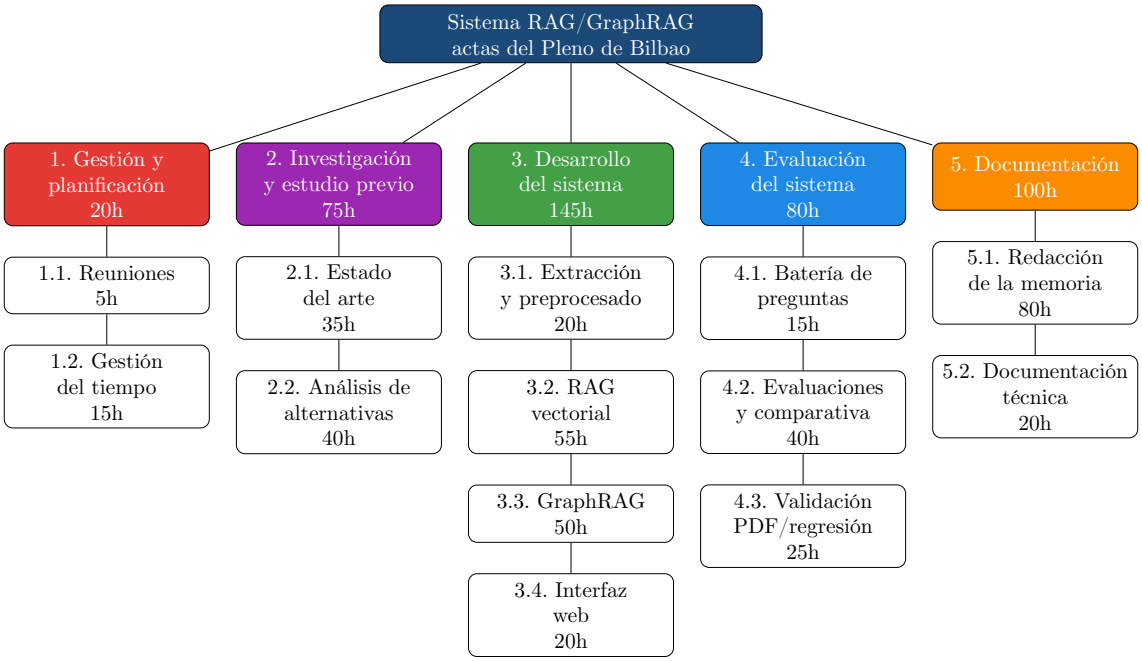


Figura 3: Diagrama EDT del proyecto

4.2. Descripción de tareas

Esta sección detalla los paquetes de trabajo del diagrama EDT. Cada uno se presenta en una tabla con su tiempo estimado, su descripción, los recursos empleados, sus predecesores y sus entregables. El color de la cabecera identifica el paquete de primer nivel al que pertenece.

4.2.1. Gestión del proyecto

Las Tablas 3 a 5 describen este paquete y sus tareas.

1. Gestión y planificación	
Tiempo estimado	20 horas
Descripción	Planificación del proyecto, seguimiento de su avance y coordinación con el director.
Recursos	Reuniones con el director y LaTeX para la elaboración de los diagramas.
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt y notas de seguimiento.

Tabla 3: Paquete de trabajo: 1. Gestión y planificación

1.1. Reuniones con el director	
Tiempo estimado	5 horas
Descripción	Reuniones presenciales de seguimiento con el director, complementadas con consultas por correo electrónico, en las que se revisa el trabajo realizado, se resuelven dudas técnicas y se acuerdan los pasos siguientes.
Recursos	Reuniones presenciales y correo electrónico.
Predecesores	-
Entregables/salidas	Observaciones del director y tareas acordadas para el periodo siguiente.

Tabla 4: Paquete de trabajo: 1.1. Reuniones con el director

1.2. Gestión del tiempo	
Tiempo estimado	15 horas
Descripción	Estimación y control del tiempo dedicado a cada paquete de trabajo, y elaboración de los diagramas EDT y Gantt.
Recursos	LaTeX, con los paquetes TikZ y pgfgantt.
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt.

Tabla 5: Paquete de trabajo: 1.2. Gestión del tiempo

4.2.2. Investigación y estudio previo

Las Tablas 6 a 8 describen este paquete y sus tareas.

2. Investigación y estudio previo	
Tiempo estimado	75 horas
Descripción	Estudio de las técnicas y tecnologías en las que se apoya el sistema, y análisis de las principales alternativas tecnológicas antes de fijar la arquitectura.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	-
Entregables/salidas	Capítulos de Contexto, de Descripción de requerimientos y análisis del estado del arte y de Análisis de alternativas de la memoria.

Tabla 6: Paquete de trabajo: 2. Investigación y estudio previo

2.1. Estudio del estado del arte	
Tiempo estimado	35 horas
Descripción	Estudio de LLM, RAG y sistemas GraphRAG existentes, y de trabajos similares sobre documentación administrativa.
Recursos	Documentación técnica y bibliografía de referencia.
Predecesores	-
Entregables/salidas	Capítulos de Contexto y de Descripción de requerimientos y análisis del estado del arte de la memoria.

Tabla 7: Paquete de trabajo: 2.1. Estudio del estado del arte

2.2. Análisis de alternativas tecnológicas	
Tiempo estimado	40 horas
Descripción	Comparación de LLM local frente a API en la nube, almacenamiento del grafo (rdflib/OWL-RL frente a una base de grafos dedicada) y estrategia de recuperación vectorial, incluyendo el diseño de la ontología OWL y el enfoque de razonamiento de GraphRAG.
Recursos	Documentación de las tecnologías evaluadas, pruebas exploratorias.
Predecesores	2.1. Estudio del estado del arte
Entregables/salidas	Capítulo de Análisis de alternativas de la memoria.

Tabla 8: Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas

4.2.3. Desarrollo del sistema

Las Tablas 9 a 13 describen este paquete y sus tareas.

3. Desarrollo del sistema	
Tiempo estimado	145 horas
Descripción	Implementación del corpus estructurado, los dos sistemas de recuperación de información y la interfaz web.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Sistema completo: corpus estructurado, Knowledge Graph, RAG vectorial, GraphRAG e interfaz web.

Tabla 9: Paquete de trabajo: 3. Desarrollo del sistema

3.1. Extracción y preprocesado del corpus	
Tiempo estimado	20 horas de trabajo activo
Descripción	Descarga automatizada de las actas del portal municipal mediante *web scraping*, extracción de los puntos del orden del día y enriquecimiento temático mediante un LLM, descritos en el Capítulo 7. No incluye el tiempo de cómputo del enriquecimiento, que se ejecuta sin supervisión y se contabiliza en la Sección 4.5.
Recursos	Python, LangChain, API de Gemini.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Corpus estructurado de puntos del orden del día, con sus metadatos.

Tabla 10: Paquete de trabajo: 3.1. Extracción y preprocesado del corpus

3.2. RAG vectorial	
Tiempo estimado	55 horas
Descripción	Implementación de la recuperación híbrida por cuatro canales, la reordenación de resultados y la generación de respuestas con cita de las fuentes.
Recursos	ChromaDB, LangChain, Cohere para la reordenación y Groq, Gemini y Ollama para la generación.
Predecesores	3.1. Extracción y preprocesado del corpus
Entregables/salidas	Sistema RAG vectorial funcional.

Tabla 11: Paquete de trabajo: 3.2. RAG vectorial

3.3. GraphRAG	
Tiempo estimado	50 horas
Descripción	Construcción del grafo RDF/OWL, razonamiento automático, traducción de la pregunta a SPARQL y corrección automática de los errores más frecuentes en las consultas generadas por el LLM.
Recursos	rdflib, owlrl, Ollama para la generación de SPARQL y Groq y Gemini para la redacción de la respuesta.
Predecesores	3.1. Extracción y preprocesado del corpus y 3.2. RAG vectorial
Entregables/salidas	Knowledge Graph y sistema GraphRAG funcional.

Tabla 12: Paquete de trabajo: 3.3. GraphRAG

3.4. Interfaz web	
Tiempo estimado	20 horas
Descripción	Integración de ambos sistemas en una interfaz web conversacional común, con enlace de cada fuente a la página del PDF. La interfaz se construye sobre Chainlit, y el trabajo se centra, por tanto, en la integración de los dos sistemas.
Recursos	Chainlit.
Predecesores	3.2. RAG vectorial, 3.3. GraphRAG
Entregables/salidas	Interfaz web funcional.

Tabla 13: Paquete de trabajo: 3.4. Interfaz web

4.2.4. Evaluación del sistema

Las Tablas 14 a 17 describen este paquete y sus tareas.

4. Evaluación del sistema	
Tiempo estimado	80 horas
Descripción	Diseño y ejecución de la evaluación comparativa de ambos sistemas, y validación de las respuestas contra las actas originales en PDF.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	3.4. Interfaz web
Entregables/salidas	Resultados de la evaluación comparativa (Capítulo 8).

Tabla 14: Paquete de trabajo: 4. Evaluación del sistema

4.1. Diseño de la batería de preguntas de referencia	
Tiempo estimado	15 horas
Descripción	Diseño de las preguntas de referencia y verificación manual de su respuesta correcta contra las actas originales.
Recursos	Lectura directa de las actas en PDF.
Predecesores	3.4. Interfaz web
Entregables/salidas	Batería de preguntas de referencia con respuesta verificada.

Tabla 15: Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia

4.2. Evaluaciones y comparativa	
Tiempo estimado	40 horas
Descripción	Ejecución de la batería sobre ambos sistemas, análisis de la calidad de recuperación y de generación, y comparativa GraphRAG frente a RAG vectorial.
Recursos	Programas de evaluación propios.
Predecesores	4.1. Diseño de la batería de preguntas de referencia
Entregables/salidas	Resultados comparativos entre ambos sistemas.

Tabla 16: Paquete de trabajo: 4.2. Evaluaciones y comparativa

4.3. Validación contra PDF real y regresión	
Tiempo estimado	25 horas
Descripción	Verificación manual del contenido de las respuestas con el texto original de las actas y ejecución de la batería de regresión tras cada cambio relevante.
Recursos	Batería de regresión propia y lectura directa de las actas en PDF.
Predecesores	4.2. Evaluaciones y comparativa
Entregables/salidas	Conjunto de casos verificados y correcciones aplicadas al sistema.

Tabla 17: Paquete de trabajo: 4.3. Validación contra PDF real y regresión

4.2.5. Documentación

Las Tablas 18 a 20 describen este paquete y sus tareas.

5. Documentación	
Tiempo estimado	100 horas
Descripción	Redacción de la memoria del TFG y de la documentación técnica del proyecto.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 18: Paquete de trabajo: 5. Documentación

5.1. Redacción de la memoria	
Tiempo estimado	80 horas
Descripción	Redacción de los capítulos de la memoria del TFG.
Recursos	LaTeX (plantilla EHU).
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 19: Paquete de trabajo: 5.1. Redacción de la memoria

5.2. Documentación técnica	
Tiempo estimado	20 horas
Descripción	Registro de las decisiones técnicas del sistema y de su justificación, y documentación de los procedimientos de instalación, de reconstrucción de los datos y de uso.
Recursos	Markdown.
Predecesores	- (paralelo al resto de paquetes)
Entregables/salidas	Documento de decisiones técnicas y guías de instalación y de reconstrucción del sistema.

Tabla 20: Paquete de trabajo: 5.2. Documentación técnica

4.3. Gestión del tiempo

4.3.1. Estimación de horas frente a duración real

La Tabla 21 compara las horas estimadas en el diagrama EDT con la dedicación real. Al no haberse llevado un registro horario detallado, las horas reales son una estimación aproximada, realizada por paquete de primer nivel al finalizar el proyecto. Las reuniones con el director han sido presenciales y puntuales, y se han complementado con consultas por correo electrónico. La gestión del tiempo se ha repartido a lo largo de todo el proyecto.

Paquete de trabajo	Estimadas	Reales (aprox.)	Desviación
1. Gestión y planificación	**20 h**	**20 h**	**0 h**
1.1. Reuniones con el director	5 h		
1.2. Gestión del tiempo	15 h		
2. Investigación y estudio previo	**75 h**	**75 h**	**0 h**
2.1. Estudio del estado del arte	35 h		
2.2. Análisis de alternativas tecnológicas	40 h		
3. Desarrollo del sistema	**145 h**	**165 h**	**+20 h**
3.1. Extracción y preprocesado del corpus	20 h		
3.2. RAG vectorial	55 h		
3.3. GraphRAG	50 h		
3.4. Interfaz web	20 h		
4. Evaluación del sistema	**80 h**	**110 h**	**+30 h**
4.1. Diseño de la batería de preguntas	15 h		
4.2. Evaluaciones y comparativa	40 h		
4.3. Validación contra PDF real y regresión	25 h		
5. Documentación	**100 h**	**105 h**	**+5 h**
5.1. Redacción de la memoria	80 h		
5.2. Documentación técnica	20 h		
Total	**420 h**	**475 h**	**+55 h**

Tabla 21: Horas estimadas frente a dedicación real aproximada

La desviación, del 13 % sobre lo estimado, se concentra sobre todo en la evaluación, y en menor medida en el desarrollo, con la misma causa en ambos casos. La planificación preveía una única evaluación comparativa al final del proyecto. En la práctica, la evaluación tuvo que repetirse varias veces con preguntas distintas, ya que cada una reveló fallos que hubo que corregir y volver a medir, como recoge la Sección 8.5, y varias de esas correcciones obligaron a ampliar y reconstruir el grafo. El coste de mano de obra de la Sección 4.5 se calcula sobre las horas reales.

4.3.2. Diagrama de Gantt

La Figura 4 muestra la planificación temporal del proyecto, desde el inicio del trabajo, el 22 de enero de 2026, hasta la entrega prevista de la memoria, a mediados de octubre de 2026. La revisión final de la memoria se prolongó hasta comienzos de noviembre. Las 420 horas estimadas en la EDT se reparten en 38 semanas. La dedicación al proyecto ha sido parcial, de forma que las horas de cada paquete se distribuyen a lo largo de varias semanas y algunos paquetes se solapan. Las tareas de gestión y la documentación técnica se desarrollan de forma continua, en paralelo al resto.

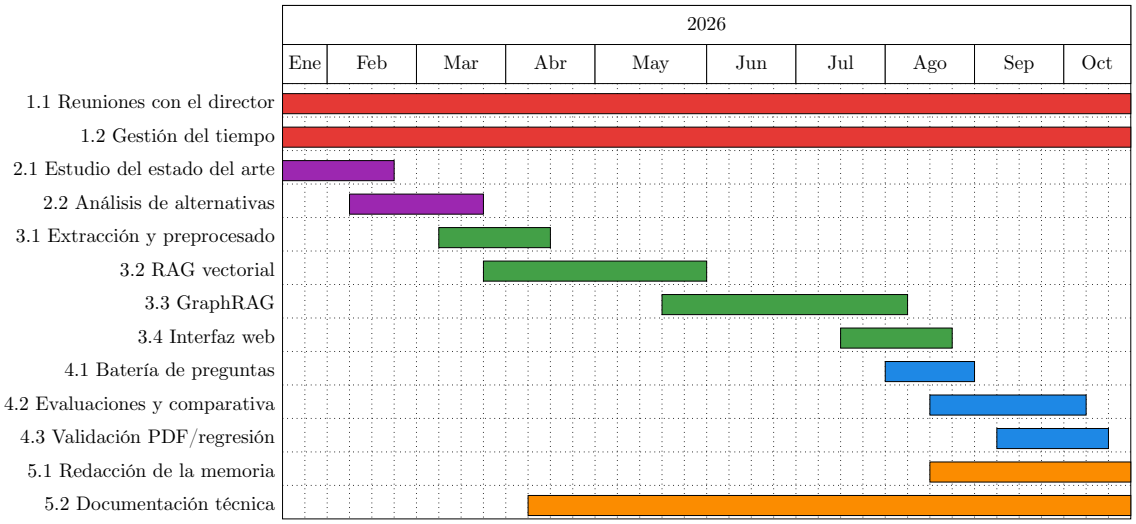


Figura 4: Diagrama de Gantt del proyecto (semanas desde el 22 de enero de 2026)

4.4. Metodología de desarrollo

El desarrollo ha seguido un enfoque iterativo, en el que cada componente se implementa, se prueba con preguntas reales y se corrige antes de abordar el siguiente, en lugar de evaluar el sistema completo solo al final. Cada iteración se apoya en una batería de regresión automática, que se ejecuta tras cualquier cambio relevante en la recuperación o en la generación de SPARQL, y en una validación manual de las respuestas con el texto original de las actas, que detecta fallos que la regresión no puede advertir, como la mezcla de contenido de dos proposiciones distintas en un bloque de respuesta bien formado. Los hallazgos y las correcciones de cada iteración se han registrado en un documento de decisiones técnicas, que ha servido de base para el Capítulo 6.

4.5. Evaluación económica

4.5.1. Costes por mano de obra

El coste de mano de obra se calcula a partir de las horas reales de dedicación, recogidas en la Tabla 21, valoradas con la tarifa de un ingeniero sin experiencia previa. Como referencia se toma el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública [22], que establece una jornada anual de 1.800 horas en su artículo 20. Su tabla salarial para 2026 [23] fija en 29.949,83 € anuales el salario de la categoría de titulado de grado superior, que se toma como equivalente a un grado universitario de 240 créditos ECTS. La tarifa resultante es de 16,64 € por hora, con la que se obtiene el coste de la Tabla 22.

Concepto	**Valor**
Horas totales reales	475 h
Tarifa por hora (29.949,83 € anuales entre 1.800 horas)	16,64 €/h
Coste de mano de obra	**7.903,43 €**

Tabla 22: Coste de mano de obra

4.5.2. Costes en hardware

El proyecto se ha desarrollado con dos equipos propios. El ordenador de sobremesa ha sido el equipo principal, y en él se han realizado el desarrollo, la evaluación y la documentación, que suman 380 horas de trabajo, junto con 65 horas de procesos de cómputo sin supervisión (primeras versiones del enriquecimiento del corpus con modelos locales y el estudio comparativo de SPARQL descrito en la Sección 5.4). El portátil se ha empleado en las 95 horas de gestión y de estudio previo. Conviene señalar que las 65 horas de cómputo desatendido no computan como dedicación del autor en la Tabla 21 ($380 + 95 = 475$ horas de trabajo humano), pero sí se imputan a la amortización del equipo al haber estado este en funcionamiento. Ambos equipos se amortizan de forma proporcional a las horas de uso dedicadas al proyecto, sobre una vida útil estimada de cuatro años, equivalente a unas 7.000 horas de uso, como recoge la Tabla 23.

Equipo	**Precio**	**Horas usadas**	**Vida útil**	**Coste**
Ordenador de sobremesa[1]	900 €	445 h	7.000 h	57 €
Portátil (Huawei, Ryzen 5)	650 €	95 h	7.000 h	9 €
			Coste de hardware	**66 €**

Tabla 23: Coste de hardware (amortizado)

4.5.3. Costes en software

Las herramientas de software empleadas son de código abierto o se han utilizado en su nivel gratuito: ChromaDB, LangChain [24], Chainlit, Ollama, Groq y el plan de prueba de Cohere. El único servicio de pago es Gemini, que se emplea en el enriquecimiento del corpus, como generador de respaldo y como juez en la evaluación con RAGAS. La partida principal de su consumo, unos 5 € en el enriquecimiento del corpus, quedó cubierta por los créditos gratuitos de bienvenida de Google Cloud, al igual que los procesos posteriores que requirieron la API, de modo que no supuso ningún desembolso, como recoge la Tabla 24.

Servicio	**Coste**
Groq (nivel gratuito)	0 €
Cohere (plan de prueba)	0 €
Ollama, ChromaDB, LangChain, Chainlit (código abierto)	0 €
Gemini (consumo cubierto por los créditos gratuitos de bienvenida)	0 €
Coste de software	**0 €**

Tabla 24: Coste de software

4.5.4. Costes indirectos

Se considera como coste indirecto el consumo eléctrico de ambos equipos durante las horas dedicadas al proyecto, a una tarifa media residencial de referencia de 0,20 €/kWh. La Tabla 25 recoge el cálculo.

[1]Precio y horas de uso estimados. Especificaciones del ordenador de sobremesa: AMD Ryzen 5 3600 (6 núcleos / 12 hilos), 16 GB de RAM, GPU NVIDIA GeForce RTX 2060 con 6 GB de VRAM, Windows 10 Pro de 64 bits.

Equipo	Consumo	Horas	Coste
Ordenador de sobremesa	~300 W	445 h	27 €
Portátil	~50 W	95 h	1 €
Coste indirecto (electricidad)			**28 €**

Tabla 25: Costes indirectos (electricidad)

4.5.5. Coste total

La Tabla 26 resume el coste total del proyecto.

Concepto	Coste
Mano de obra	7.903,43 €
Hardware (amortizado)	66 €
Software	0 €
Indirectos (electricidad)	28 €
Coste total	**7.997,43 €**

Tabla 26: Coste total del proyecto

La mano de obra supone en torno al 99 % del coste total, como corresponde a un proyecto que no requiere hardware especializado y cuyo único servicio de pago quedó cubierto por créditos gratuitos.

4.6. Análisis de riesgos

Se han identificado cinco riesgos. Para cada uno se indica su descripción, las medidas de prevención y de contingencia previstas, su probabilidad y su impacto, en las Tablas 27 a 31.

1. Dependencia de APIs externas (Groq)	
Descripción	El sistema usa la API de Groq para generar las respuestas finales; si el servicio falla o se agota la cuota diaria, la calidad de la respuesta puede degradarse.
Plan de prevención	Vigilar el consumo de cuota antes de lanzar baterías de pruebas extensas.
Plan de contingencia	Paso automático a Gemini cuando Groq no responde o se agota la cuota, y a un modelo local servido con Ollama si Gemini también falla.
Probabilidad	Media
Impacto	Bajo: el sistema sigue funcionando, con menor calidad de redacción.

Tabla 27: Riesgo: 1. Dependencia de APIs externas

2. Errores silenciosos en la generación de SPARQL	
Descripción	El LLM puede generar una consulta SPARQL sintácticamente válida pero semánticamente incorrecta, por ejemplo con un patrón que cuenta de más, una propiedad inexistente o un filtro omitido. El resultado es una cifra errónea que se presenta como un dato del grafo, sin ningún error visible.
Plan de prevención	Corrección automática de los errores conocidos antes de ejecutar la consulta, descrita en el Capítulo 6, y validación manual de las respuestas con el acta original en cada ronda de pruebas.
Plan de contingencia	Batería de regresión automática, que se ejecuta tras cualquier cambio en la generación de SPARQL para detectar si una corrección reintroduce un fallo ya resuelto.
Probabilidad	Alta: es el fallo más frecuente de los encontrados durante el desarrollo.
Impacto	Alto: una cifra incorrecta sin error visible es el caso más grave para un sistema cuyo requisito central es la trazabilidad, recogida en el requisito RNF1.

Tabla 28: Riesgo: 2. Errores silenciosos en la generación de SPARQL

3. Calidad variable del texto extraído de las actas	
Descripción	Las actas más antiguas del corpus presentan defectos en el texto que se extrae del PDF, como espacios intercalados, palabras cortadas y nombres escritos indistintamente en euskera y en castellano, y cuatro actas están escaneadas y deben transcribirse. Estos defectos dificultan la extracción de proposiciones y la identificación de concejales.
Plan de prevención	Unificación de los nombres de concejal mediante la normalización fonética de los apellidos, la comparación de los nombres de pila y una tabla de equivalencias entre las formas en euskera y en castellano.
Plan de contingencia	Cuando la resolución automática no alcanza la confianza suficiente, la entidad se deja sin resolver: se prefiere no introducir un dato falso a completar todos los casos.
Probabilidad	Media, concentrada en las actas más antiguas del corpus.
Impacto	Medio: afecta a la cobertura del voto nominal y a la atribución de partido de algunos concejales, no a la fiabilidad de los datos que sí se resuelven.

Tabla 29: Riesgo: 3. Calidad variable del texto extraído de las actas

4. Plazo de entrega ajustado con un único desarrollador	
Descripción	El proyecto lo desarrolla una sola persona, con entrega prevista a mediados de octubre de 2026, como recoge la Sección 4.3.2. Ante un imprevisto de salud, técnico o de disponibilidad no es posible reasignar el trabajo.
Plan de prevención	Seguimiento semanal del avance respecto al diagrama de Gantt de la Figura 4 y prioridad de los requisitos funcionales de la Tabla 1 frente a las mejoras no esenciales.
Plan de contingencia	Reducción del alcance del trabajo, por ejemplo del tamaño de la batería de preguntas de referencia, antes que renunciar a la trazabilidad o a la corrección de los datos.
Probabilidad	Media.
Impacto	Alto: sin margen de reasignación, un imprevisto afecta directamente a la fecha de entrega.

Tabla 30: Riesgo: 4. Plazo de entrega ajustado con un único desarrollador

5. Nombres y grupos ambiguos o mal atribuidos en las actas	
Descripción	Las actas identifican a los concejales por su apellido, y varios lo comparten dentro del censo, como ocurre con tres concejales apellidados Gil en mandatos distintos. Sin desambiguar, el voto nominal puede atribuirse a la persona equivocada. Del mismo modo, una palabra común puede confundirse con el nombre de un grupo político, como sucede con el término ciudadanos y el partido Ciudadanos.
Plan de prevención	Resolución de cada apellido con el censo de concejales del mandato correspondiente, y comparación exacta del nombre del grupo político con sus denominaciones reconocidas, en lugar de una coincidencia de texto libre.
Plan de contingencia	Cuando varios concejales del censo comparten apellido y grupo en la misma fecha, se registra únicamente el voto agregado por grupo y se omite el voto nominal de esa persona.
Probabilidad	Media, concentrada en los apellidos comunes.
Impacto	Medio: afecta a la atribución de voto nominal a personas concretas, no al recuento agregado por grupo, el dato que más preguntas de agregación utilizan.

Tabla 31: Riesgo: 5. Nombres y grupos ambiguos o mal atribuidos en las actas

5. Análisis de alternativas

Este capítulo analiza las principales alternativas tecnológicas consideradas antes de fijar la arquitectura del sistema. Para cada decisión se exponen las ventajas y los inconvenientes de las opciones valoradas y la elección adoptada. Las tres primeras secciones tratan los LLM, el almacenamiento del grafo y la recuperación vectorial. La última presenta el estudio comparativo realizado para elegir el modelo y la forma del *prompt* en la generación de SPARQL.

5.1. LLM: ejecución local frente a API en la nube

El sistema emplea LLM en dos situaciones distintas. La generación interactiva produce una respuesta por cada pregunta del usuario, en tiempo real. El enriquecimiento por lotes clasifica una sola vez, sin supervisión, todos los puntos del orden del día del corpus. Para ambas se valoraron tres opciones: modelos locales servidos con Ollama [25], la API de Groq [26] en su nivel gratuito y la API de pago de Gemini [27].

5.1.1. Puntos positivos

- **Ollama, en local.** No tiene coste por *token* ni límite de cuota, y el procesamiento no depende de un tercero.
- **Groq.** Su nivel gratuito ofrece una inferencia muy rápida, que permite servir respuestas interactivas con el modelo gpt-oss-120b [28] sin disponer de GPU propia.
- **Gemini.** El modelo Gemini 3.5 Flash-Lite tiene un coste bajo y predecible, de 0,30 $ por millón de *tokens* de entrada y 2,50 $ por millón de *tokens* de salida. El enriquecimiento de la primera extracción, de unos 3.400 registros, consumió unos 5 €, cubiertos por los créditos gratuitos de bienvenida.

5.1.2. Puntos negativos

- **Ollama, en local.** Sin GPU, la generación es inviable en los procesos por lotes, pues a unos 200 segundos por proposición en un portátil, la extracción de los 3.400 registros de la primera versión habría requerido más de 180 horas. Exige un equipo con GPU, que en este proyecto ha sido el ordenador de sobremesa (Sección 4.5).
- **Groq.** El nivel gratuito limita el consumo del modelo empleado a 200.000 *tokens* al día. La cuota está asociada a la cuenta, por lo que no se evita cambiando de equipo ni de clave, y no basta para enriquecer el corpus completo de una vez.
- **Gemini.** Es de pago desde el primer *token*, lo que obliga a controlar el gasto, y los modelos disponibles para una misma clave cambian con el tiempo, hasta el punto de que dos de los previstos inicialmente dejaron de estar disponibles durante el desarrollo.

5.1.3. Conclusión

No se elige una única opción, sino una combinación según el uso. La generación interactiva se confía a Groq, por su rapidez y su gratuidad, con Gemini y un modelo local como respaldo ante un fallo o el agotamiento de la cuota. El enriquecimiento por lotes se realiza con Gemini 3.5 Flash-Lite, cuyo coste es bajo y que evita las más de 180 horas que habría requerido un modelo local sin GPU. Los modelos locales con GPU se emplean en la generación de SPARQL del sistema GraphRAG y en el estudio comparativo de la Sección 5.4, que repite el mismo experimento cientos de veces por modelo, sin coste por consulta. La elección de un modelo abierto ejecutado en local para generar consultas tiene respaldo en la literatura sobre *text-to-SQL*, según la cual los modelos abiertos de tamaño moderado son competitivos en esta tarea cuando se combinan con una buena construcción del *prompt* o se ajustan a ella [17, 18].

5.2. Almacenamiento del grafo: librería en memoria frente a base de grafos dedicada

El Knowledge Graph, de unos 660.000 triples una vez materializados los que deduce el razonador OWL-RL, puede almacenarse y consultarse con una librería en memoria, como rdflib [29] junto con owlrl [30], o con un motor de grafos desplegado como servicio, como GraphDB [31], Apache Jena Fuseki [32] o Blazegraph [33].

5.2.1. Puntos positivos

- **rdflib y owlrl.** No requieren desplegar ni mantener un servicio aparte, ya que el grafo es un fichero que se carga en memoria al arrancar, lo que simplifica la instalación y responde al requisito de coste RNF3. El razonamiento OWL-RL se ejecuta una sola vez, al construir el grafo, a diferencia de los motores que razonan al ejecutar cada consulta.
- **Base de grafos dedicada.** Ofrece índices optimizados para consultas SPARQL sobre grafos de gran tamaño, gestión de la concurrencia entre usuarios y herramientas de administración y de inspección visual.

5.2.2. Puntos negativos

- **rdflib y owlrl.** El grafo completo se carga en memoria en cada arranque, lo que tarda unos 20 segundos, y las consultas complejas son más lentas que con un motor dotado de índices persistentes. Además, rdflib no permite cancelar una consulta en curso, por lo que el sistema limita a 20 segundos el tiempo de cada una. Con el volumen actual, las consultas habituales se resuelven en menos de un segundo, pero es una solución que no escala indefinidamente.
- **Base de grafos dedicada.** Añade una pieza de infraestructura que instalar, configurar y mantener, con un coste de licencia o de alojamiento en las opciones no libres, para un proyecto con un único desarrollador y un grafo que cabe en memoria.

5.2.3. Conclusión

Se descarta la base de grafos dedicada porque el volumen del grafo no la justifica y rdflib [29] con owlrl [30] satisface los requisitos RF2 y RF8 sin coste adicional de infraestructura. Un motor dedicado pasaría a ser preferible si el corpus creciera en varios órdenes de magnitud o si el sistema tuviera que atender consultas concurrentes de muchos usuarios con baja latencia.

5.3. Recuperación vectorial: búsqueda híbrida frente a búsqueda semántica

El RAG vectorial puede limitarse a la búsqueda semántica por *embeddings* o combinarla con otros canales de recuperación sobre el mismo almacén, que en este trabajo es ChromaDB [34].

5.3.1. Puntos positivos

- **Solo semántica.** La arquitectura es más simple, con un único canal de recuperación y sin lógica de fusión de resultados.
- **Híbrida.** Cada canal cubre un fallo de los demás. El canal literal recupera las menciones únicas de nombres propios que el *embedding* no aproxima lo suficiente, como una empresa citada una sola vez. El canal temático recupera fragmentos relevantes para preguntas generales sobre un tema amplio, que no comparten vocabulario con ellos, porque compara la clasificación temática y no el texto. El canal de proposiciones, añadido con posterioridad, busca por significado en el resumen de cada proposición y recupera las que no emplean las palabras de la pregunta (Sección 8.3.2).

5.3.2. Puntos negativos

- **Solo semántica.** Pierde precisión en los casos que motivan los canales adicionales, las menciones únicas y las preguntas temáticas amplias, con el riesgo de responder que no hay información cuando el corpus sí la contiene.
- **Híbrida.** Genera más candidatos que fusionar y ordenar en cada pregunta, lo que exige una etapa de reordenación con un modelo *cross-encoder* [13] para que el mayor volumen de candidatos no degrade la relevancia de los primeros resultados, y aumenta la complejidad de implementación y de depuración.

5.3.3. Conclusión

Se adopta la recuperación híbrida sobre ChromaDB. Su mayor coste de implementación se compensa con la cobertura de preguntas que la búsqueda semántica no resuelve por sí sola, como las menciones únicas y las preguntas temáticas amplias, lo que reduce los casos en que el sistema responde que no dispone de información cuando el corpus sí la contiene. La aportación del canal de proposiciones se mide en la Sección 8.3.2.

5.4. Modelo y *prompt* para la generación de SPARQL: estudio comparativo

El componente más delicado del sistema GraphRAG es el que convierte la pregunta en una consulta SPARQL, como se expone en la Sección 3.2.2. El modelo y la forma del *prompt* se han elegido a partir de un estudio propio sobre el grafo del proyecto.

5.4.1. Método

Se prepararon 35 preguntas de referencia de nueve tipos: conteo, ranking, proporción, evolución temporal, persona, voto nominal, entidad, total y coautoría. Cada pregunta lleva asociado el resultado que debe devolver. Una consulta generada se considera correcta si se ejecuta sin error y sus filas contienen ese resultado, con lo que se mide la precisión de ejecución y no el parecido de la consulta con una de referencia. Las demás consultas se clasifican como erróneas, si devuelven otro dato, vacías, si no devuelven ninguno, o con error, si no se ejecutan.

Se compararon diez modelos y varias formas de construir el *prompt*, de menor a mayor información: solo el esquema del grafo; el esquema con el vocabulario real, formado por los identificadores de los temas y de los grupos; el *prompt* completo, con reglas y ejemplos fijos o elegidos para cada pregunta; y un *prompt* compacto con tres ejemplos elegidos para cada pregunta, por coincidencia de palabras o por similitud de *embeddings*. En una última variante, la consulta generada se somete además a una corrección automática de los errores más frecuentes. Cada combinación se repitió hasta tres veces sobre las 35 preguntas, con un total de 3.331 ejecuciones.

5.4.2. Resultados

La Tabla 32 muestra el efecto de la forma del *prompt* con dos de los modelos, y la Tabla 33, el mejor resultado de cada modelo.

Forma del *prompt*	**Qwen2.5-7B** [35]	**Gemini 3.5 Flash-Lite**
Solo el esquema del grafo	30 %	61 %
Esquema y vocabulario real	38 %	–
Completo, con reglas y ejemplos fijos	76 %	86 %
Compacto, con ejemplos por coincidencia de palabras	73 %	96 %
Compacto, con ejemplos por *embeddings*	71 %	95 %
Con ejemplos por *embeddings* y corrección posterior	77 %	97 %
Completo, con ejemplos por coincidencia de palabras y corrección posterior	80 %	–

Tabla 32: Consultas correctas según la forma del *prompt* (35 preguntas, tres repeticiones). Las celdas con – corresponden a configuraciones no medidas con ese modelo

Modelo	Ejecución	Correctas	n
Gemini 3.5 Flash-Lite	Nube	97 %	105
Qwen3:8B	Local	94 %	105
gpt-oss-20b (Groq)	Nube	86 %	35
Qwen2.5-Coder-7B	Local	85 %	105
Qwen2.5-14B	Local	83 %	70
Qwen2.5-7B	Local	80 %	105
Mistral-7B	Local	66 %	35
Llama 3.2	Local	43 %	105
Gemma3-4B	Local	40 %	35
gpt-oss-120b (Groq)	Nube	no concluyente	–

Tabla 33: Consultas correctas con la mejor forma de *prompt* de cada modelo; n es el número de consultas generadas

Las dos tablas permiten cuatro observaciones, sobre la forma del *prompt*, la selección de los ejemplos, el tamaño del modelo y el tipo de pregunta.

La forma del *prompt* influye más que el modelo. Con el mismo modelo de 7B, pasar de proporcionar solo el esquema a incluir reglas y ejemplos eleva las consultas correctas del 30 % al 76 %. El esquema por sí solo es insuficiente incluso para el mejor modelo, que se queda en el 61 %, en línea con lo que recoge la literatura [15].

La selección de los ejemplos por similitud mejora claramente a los modelos más capaces, y en los modelos pequeños su efecto depende de la forma del *prompt*. Mejora los resultados de Gemini, del 86 % al 96 %, y de Qwen3 [36], del 83 % al 91 % (alcanzando el 94 % de la Tabla 33 al incorporar la corrección sintáctica posterior mediante guardas), pero no los del modelo de 7B, que se mantiene o empeora ligeramente respecto a los ejemplos fijos, del 76 % al 71–73 %. En la evaluación de Gao et al., en cambio, esa selección mejora los cuatro modelos probados [17].

El tamaño del modelo presenta rendimientos decrecientes. Con el *prompt* completo, el modelo de 14B obtiene el mismo 83 % que el de 8B de una generación posterior, y tarda más del triple por consulta. Un modelo local de 8B queda, además, a tres puntos del mejor modelo de la nube.

En cuanto al tipo de pregunta, las proporciones son las más difíciles. Piden un porcentaje, lo que exige dos recuentos en una misma consulta, y aunque son pocas entre las 35 preguntas, explican la mitad de las consultas incorrectas del modelo elegido.

El resultado de gpt-oss-120b no es concluyente. La cuota del nivel gratuito de Groq interrumpió sus ejecuciones, y 175 de sus 205 llamadas terminaron con un error de límite de uso. De las 27 que respondió con la forma compacta del *prompt*, 24 fueron correctas, pero una muestra tan reducida no permite compararlo con el resto.

5.4.3. Conclusión

Se elige Qwen3:8B, ejecutado en local, con tres ejemplos seleccionados por *embeddings* para cada pregunta y corrección posterior de la consulta, la configuración con la que obtuvo su mejor resultado. Queda a tres puntos del mejor modelo probado, no tiene coste

por consulta ni límite de cuota y tarda entre 12 y 15 segundos por consulta en el equipo disponible. Gemini, que responde en menos de un segundo, se reserva como segunda opinión cuando la consulta generada en local falla.

El estudio tiene tres limitaciones. Con 35 preguntas, una sola pregunta modifica el resultado en casi tres puntos, de ahí que las diferencias pequeñas entre modelos no sean significativas. Un 94 % de consultas correctas implica, además, un 6 % de incorrectas. La mayoría son errores visibles, pero algunas se ejecutan y devuelven un dato equivocado sin señal de fallo, lo que motiva las guardas deterministas y la decisión de que el LLM escriba la consulta solo cuando el programa no puede componerla, descritas en el Capítulo 6. Por último, la comparación entre los diez modelos se realizó sobre una versión anterior del grafo. Tras las ampliaciones posteriores, la configuración elegida se repitió con el grafo y el código de la versión entregada, con las respuestas de referencia recalculadas, y obtuvo un 97 % de consultas correctas en 105 ejecuciones, un resultado coherente con el del estudio.

6. Descripción de la solución propuesta

Este capítulo describe el sistema desarrollado: su arquitectura general, la construcción del Knowledge Graph, los dos sistemas de recuperación de información, la generación de las respuestas, la interfaz web y las principales decisiones de configuración. La obtención y la preparación del corpus sobre el que opera se describen en el Capítulo 7.

6.1. Arquitectura general del sistema

El trabajo comprende dos sistemas de pregunta-respuesta construidos sobre los mismos documentos, como muestra la Figura 5. Ambos parten de las actas, se consultan desde una misma interfaz conversacional y comparten varias piezas intermedias, pero almacenan la información en dos estructuras de datos distintas. El RAG vectorial conserva el texto de las actas dividido en fragmentos y los recupera por su similitud con la pregunta. El GraphRAG conserva en un Knowledge Graph los hechos extraídos de las actas y los consulta con SPARQL. El hecho de que ambos partan de los mismos documentos y de los mismos datos de enriquecimiento es lo que permite compararlos en el Capítulo 8.

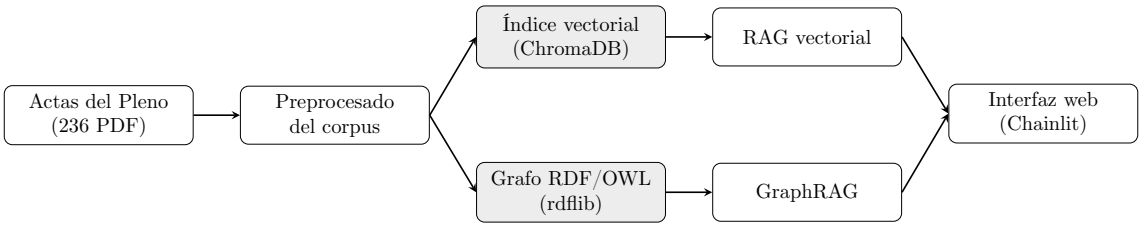


Figura 5: Arquitectura general: dos estructuras de datos sobre los mismos documentos

El código sigue la misma división. Un módulo común reúne las rutas de los datos, el acceso a los LLM con sus respaldos y la normalización de los nombres de los grupos políticos, que utilizan los dos sistemas. Cada sistema tiene su propio módulo, organizado por etapas. El del RAG vectorial va del PDF a los fragmentos, de la pregunta a los fragmentos y de los fragmentos a la respuesta, mientras que el del grafo se divide en construcción y consulta. La interfaz se limita a enviar la pregunta al sistema elegido y a mostrar lo que este devuelve.

6.2. Construcción del Knowledge Graph

El grafo se construye a partir de los puntos del orden del día extraídos y enriquecidos, cuyo proceso describe el Capítulo 7, en cuatro pasos.

Ontología. Una ontología OWL propia define el contenido posible del grafo mediante 18 clases y 81 propiedades. Las clases principales son el punto del orden del día y la proposición, que es un tipo de punto; el pleno y la legislatura en los que se trata; el grupo político y el concejal; la votación, la enmienda y la intervención de un orador; las entidades mencionadas, que pueden ser organizaciones, lugares o personas; y los barrios y los distritos de la ciudad. Las propiedades enlazan esas clases e indican, por ejemplo, quién presenta una proposición, qué grupos votan a favor o en contra, de qué tema trata o en qué página del acta se encuentra. Con ese vocabulario se escriben después las consultas. La Figura 6 muestra las clases principales y las propiedades que las relacionan.

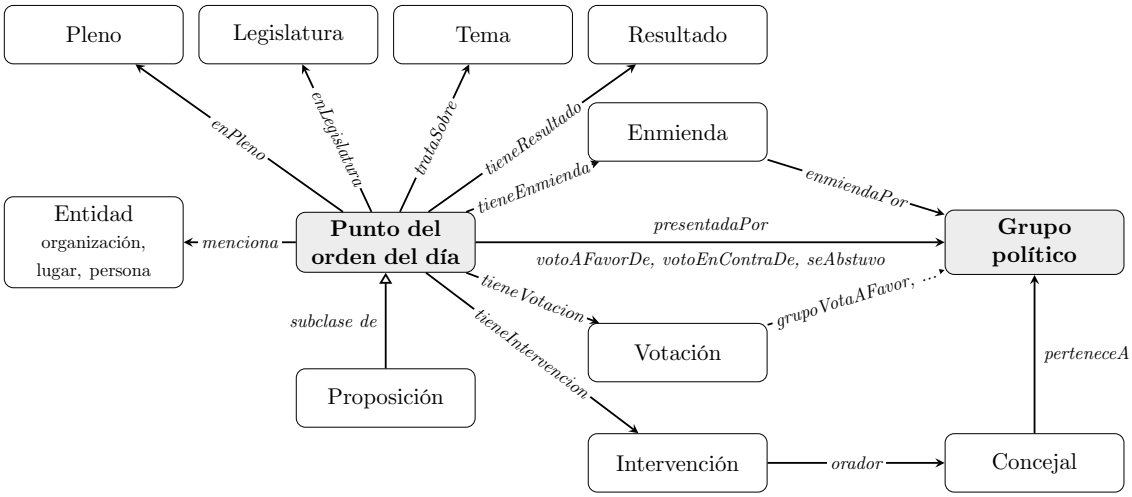


Figura 6: Clases principales de la ontología y propiedades que las relacionan

Taxonomía de temas. Los temas se organizan en una taxonomía SKOS de dos niveles con 58 conceptos: 19 temas generales, como vivienda, movilidad y transporte o presupuestos y fiscalidad, y sus subtemas, como desahucios, bicicleta o impuestos. Cada concepto incluye sus sinónimos en castellano y en euskera. Es la misma taxonomía que emplea el canal temático del RAG vectorial.

Razonamiento. Sobre el grafo se ejecuta una sola vez, al construirlo, un razonador OWL-RL que añade los hechos que se deducen de la ontología. El más útil es la herencia temática, por la cual una proposición clasificada en el subtema de los desahucios pasa a tratar también del tema general de la vivienda, de modo que una pregunta sobre vivienda la incluye sin que la consulta tenga que enumerar los subtemas. El razonador deduce también la legislatura de cada punto a partir de su pleno, el distrito a partir del barrio y la relación inversa entre un punto y sus intervenciones. El razonamiento no se aplica al ejecutar cada consulta, ya que rdflib evalúa las consultas sobre los triples presentes en el grafo. Se realiza por *forward chaining*, estrategia en la que las reglas de OWL-RL se aplican sobre los datos hasta que no se deduce ningún hecho nuevo. Los hechos deducidos se materializan, de modo que se añaden al grafo como nuevos triples y se guardan con él. El grafo pasa así de unos 562.000 triples afirmados a unos 660.000, una vez descartada una parte de las deducciones triviales del razonador, como la identidad de cada recurso consigo mismo. La cifra final se recoge en la Tabla 36.

Validación. Un conjunto de restricciones SHACL, independiente del código que construye el grafo, comprueba su integridad, tal como exige el requisito RF8. Verifica, entre otras condiciones, que toda proposición tenga una fecha, un título y un único grupo proponente, que su resultado sea uno de los seis valores previstos, que son los cinco

resultados posibles de una votación y la ausencia de resultado, y que quien vota sea un grupo o un concejal del censo. La versión entregada no presenta ninguna violación.

Entre la extracción y el razonamiento se aplica además una capa de correcciones deterministas, resultado de las auditorías realizadas con las actas. Esta capa clasifica cada punto por su tipo, que puede ser proposición de un grupo, propuesta del gobierno, iniciativa vecinal, pregunta o dación de cuenta, en la que el gobierno informa al Pleno de un asunto sin que se vote, entre otros; separa las votaciones de un mismo punto y marca cuál es la decisiva; corrige los recuentos que no coinciden con el acta; y asigna a cada punto su pleno y su legislatura. Solo los puntos que son proposiciones entran en los recuentos.

Texto de los debates. El grafo no almacena el texto íntegro de las intervenciones, sino un resumen de cada una y la postura del orador. El texto de los debates, de unos 95 millones de caracteres, ya está en el índice vectorial, y copiarlo al grafo habría duplicado ese índice en lugar de ofrecer una estructura distinta con la que compararlo. Además, habría más que duplicado el tamaño de un grafo que se carga completo en memoria. La otra posibilidad, convertir en datos estructurados todo lo que se dice en cada intervención, habría exigido que un LLM extrajera muchos más datos, y cada dato extraído por un LLM, que no es determinista, es una posible fuente de error, como se analiza en la Sección 8.7. El grafo recoge, por tanto, quién intervino en cada debate, con qué postura y con qué argumentos, de forma resumida, junto con la página del acta en la que puede leerse el texto original. Una tercera posibilidad, que no exige copiar el texto ni extraer más datos de él, consiste en enlazar cada elemento del grafo con los fragmentos del acta de los que procede. Se recoge como trabajo futuro en la Sección 9.5.

La Figura 7 muestra el resultado para una proposición concreta, la misma cuyo comienzo en el acta reproduce la parte derecha de la Figura 11. Los triples recogen el pleno y la legislatura en los que se debatió, el grupo que la presentó, sus temas, una de las entidades que menciona, el voto de cada grupo y la página del acta en la que comienza. Las flechas discontinuas señalan los dos triples que materializa el razonador, el de la legislatura, que se deduce del pleno, y el del tema general de la vivienda, que se deduce del subtema del alquiler. La proposición, que pide definir como gran tenedor a quien posea cinco o más inmuebles residenciales, se aprobó con el voto en contra de un grupo.

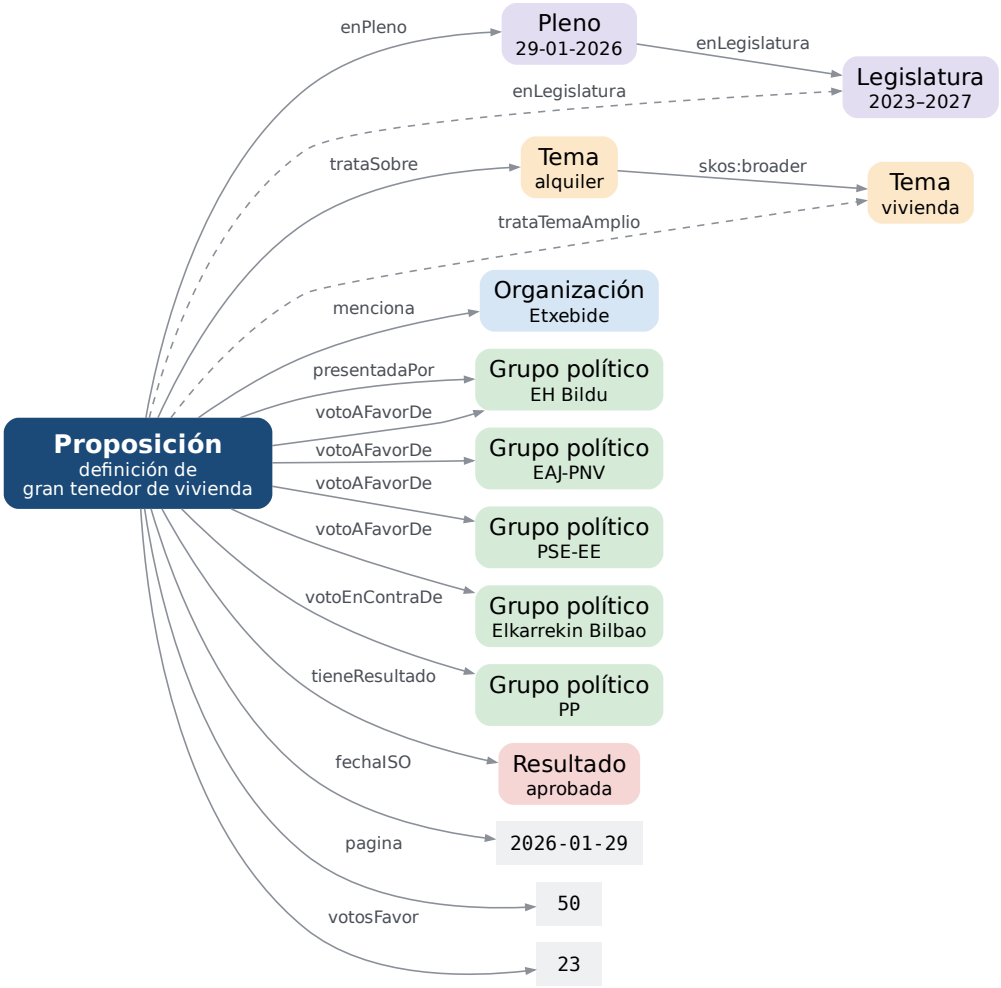


Figura 7: Selección de los triples de una proposición del grafo. Las flechas discontinuas son triples materializados por el razonador y los rectángulos grises, valores literales

6.3. Sistema de recuperación de información

6.3.1. RAG vectorial

Indexado. El índice se construye como describe la Sección 7.4. Contiene 97.217 fragmentos de 1.200 caracteres, cada uno con la fecha del pleno, el título del punto, el orador y su grupo, la página del PDF, el resultado de la votación y los datos de enriquecimiento de su proposición, los mismos que contiene el grafo. Un segundo índice, de menor tamaño, almacena un documento breve por cada proposición con resumen, con su título, su grupo, sus temas y su resumen.

Recuperación. Ante una pregunta, el sistema realiza los pasos siguientes:

1. **Analiza la pregunta** con reglas. Si menciona una fecha, un año o un grupo político, esos datos se utilizan como filtros.
2. **Genera tres variantes** de la pregunta con un LLM (*MultiQuery*), para no depender de una única redacción.

3. **Busca por cuatro canales**, presentados en la Sección 5.3: el semántico, por similitud de significado; el literal, que busca las palabras menos frecuentes de la pregunta tal como están escritas; el temático, que recupera proposiciones clasificadas en el tema de la pregunta; y el de proposiciones, que busca por significado en los resúmenes.
4. **Reordena** los candidatos de los cuatro canales con el modelo de reordenación de Cohere, o con el reordenador local de respaldo descrito en la Sección 8.3.2, y conserva los mejores.
5. **Compone el contexto.** Agrupa los fragmentos elegidos por pleno y proposición, añade el debate de las proposiciones elegidas y reparte entre los bloques un límite de 20.000 caracteres.

Si ningún candidato supera un umbral mínimo de relevancia, la pregunta se considera ajena a las actas y el sistema lo indica, en lugar de responder con el escaso material encontrado, conforme al requisito RNF2.

6.3.2. GraphRAG

El GraphRAG debe convertir la pregunta en una consulta SPARQL. Dado que una consulta errónea se ejecuta sin error y devuelve una cifra falsa, como se expone en la Sección 3.2.2, el diseño reduce al mínimo los casos en que la consulta la escribe un LLM. Para cada pregunta se prueban, por este orden, tres vías:

1. **Consulta compuesta por el programa.** Un análisis con reglas extrae de la pregunta el tema, los grupos, los años, el resultado y la forma de lo que se pide, que puede ser un recuento, un ranking por grupo o por año, una votación, la última vez que se trató un asunto o un listado. Un LLM se limita a señalar cuál es el asunto concreto. Si la forma es una de las previstas, el programa compone la consulta con esas piezas, sin que ningún LLM escriba SPARQL.
2. **Plantilla.** La pregunta se compara, por similitud de *embeddings*, con las frases de ejemplo de un catálogo de formas de consulta, entre ellas el conteo, el ranking por grupo, la proporción y la evolución por años. Si se parece lo suficiente a una de ellas, se rellena su plantilla con los valores de la pregunta.
3. **Consulta generada por un LLM.** En los demás casos, un modelo local, Qwen3:8B, escribe la consulta a partir de la descripción del grafo y de varios ejemplos. Antes de aceptarla, unas guardas deterministas comprueban que solo utiliza términos existentes en el grafo, corrigen los errores más frecuentes y verifican que filtra todo lo que pide la pregunta. En caso contrario, se pide al modelo que la corrija, hasta dos veces, y como último recurso se solicita una segunda opinión a otro modelo.

Cuando ninguna vía produce una consulta completa, o el asunto de la pregunta no aparece en ninguna proposición, el sistema responde que el dato no consta o que no ha podido componer la consulta, en lugar de redactar una respuesta a partir de filas que no corresponden a lo preguntado. La diferencia de fiabilidad entre la primera vía y la tercera se mide en la Sección 8.4.

6.4. Generación de respuestas

En los dos sistemas, la respuesta la redacta un LLM a partir de lo recuperado, y las fuentes las añade el programa, no el LLM.

En el RAG vectorial, el modelo recibe los bloques del contexto, cada uno identificado con la fecha de su pleno y el título de su punto, y redacta una crónica con un bloque por proposición, que incluye la fecha, el grupo proponente, el título, lo que se propone, un resumen del debate y el resultado de la votación. A continuación, el programa depura la respuesta, eliminando los bloques repetidos o vacíos, y coloca bajo cada bloque el enlace a la página del acta de la que procede, a partir de los metadatos de los fragmentos. Un LLM al que se le pide que cite la página tiende a inventarla. Como aquí la página no la escribe el modelo, el enlace se construye a partir de los metadatos y conduce a un documento existente, lo que satisface el requisito RF6. La Figura 8 muestra una respuesta de este sistema.

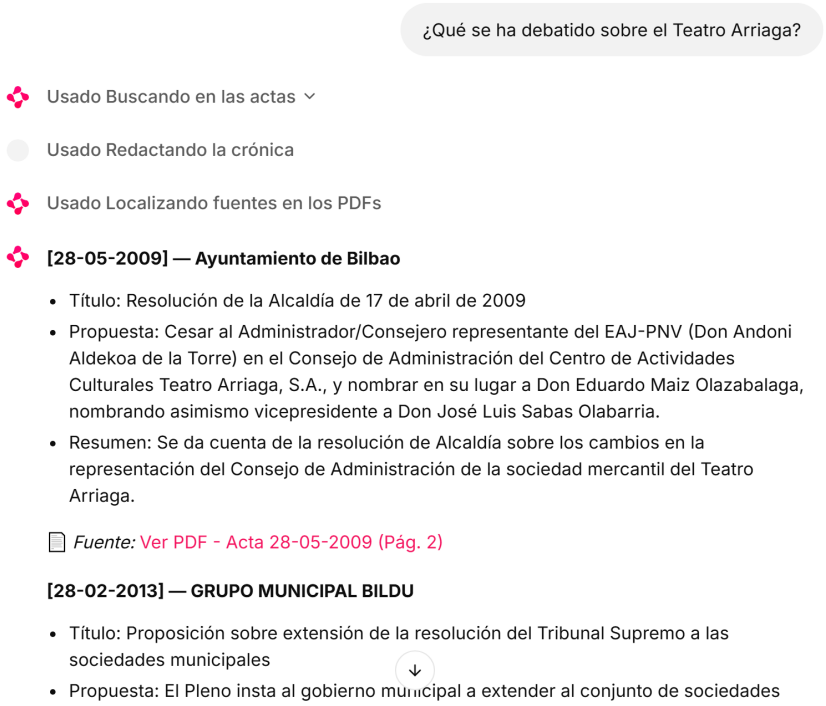


Figura 8: Respuesta del RAG vectorial, con el enlace a la página del acta bajo cada bloque

En el GraphRAG, el modelo recibe las filas devueltas por la consulta, la propia consulta, para que conozca los filtros ya aplicados, e indicaciones sobre cómo interpretar los datos según el tipo de pregunta. Su función es explicar las filas, no calcular, puesto que las cifras proceden del grafo. Si la respuesta cita proposiciones concretas, el programa añade el enlace a su página. Si es una cifra agregada, se indica que se ha calculado sobre el grafo y que no corresponde a un único documento. La Figura 9 muestra una respuesta de este sistema, junto con la consulta ejecutada.

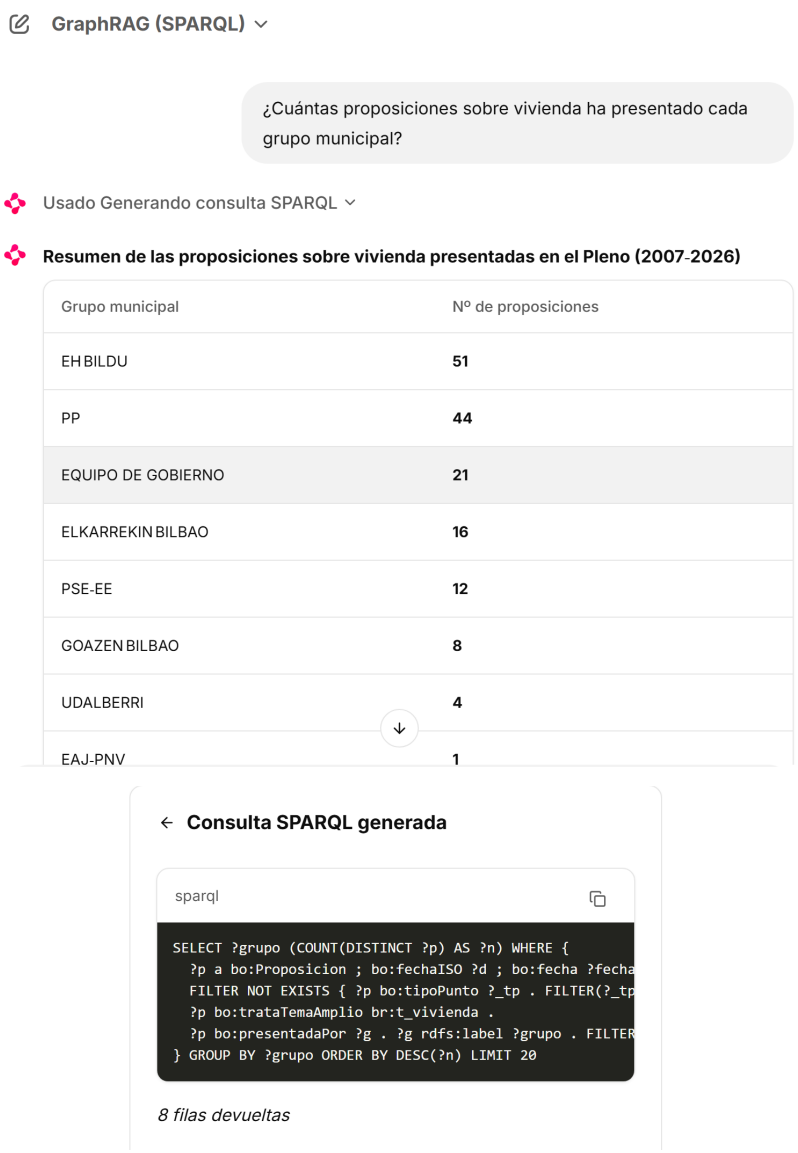


Figura 9: Respuesta del GraphRAG y consulta SPARQL ejecutada. La interfaz muestra la consulta en un panel lateral, que aquí se ha colocado debajo

6.5. Interfaz web del sistema

La interfaz es una aplicación web conversacional construida con Chainlit [37]. El usuario elige uno de los dos perfiles, RAG Vectorial o GraphRAG (SPARQL), y escribe su pregunta o selecciona una de las preguntas de ejemplo que se ofrecen al inicio. Mientras se prepara la respuesta, la interfaz muestra los pasos en curso, desde la búsqueda en las actas hasta la localización de las fuentes, y permite desplegarlos para consultar su detalle.

En la Figura 8 se aprecia la forma de la respuesta del RAG vectorial, con un bloque por proposición, cada uno con el enlace que abre el PDF del acta en la página correspondiente. En la Figura 9, la respuesta del GraphRAG incluye la tabla del recuento por grupo, y la interfaz presenta en un panel la consulta SPARQL ejecutada y el número de filas devueltas, de modo que quien sepa leerla puede comprobar qué se ha contado.

6.6. Flujo completo del sistema

La Figura 10 resume el recorrido de una pregunta en cada uno de los dos sistemas, desde que el usuario la escribe hasta que recibe la respuesta.

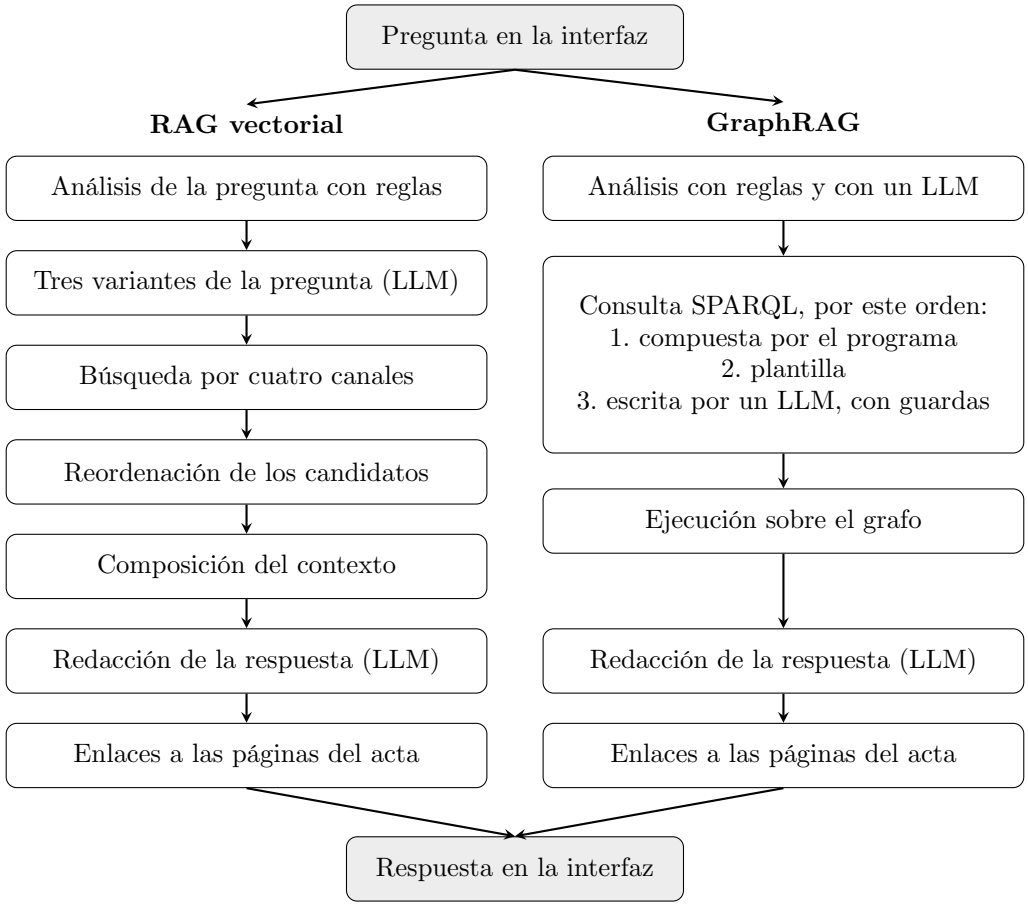


Figura 10: Recorrido de una pregunta en cada uno de los dos sistemas

El recorrido se ilustra con una pregunta de cada tipo. Ante la pregunta "¿Qué se ha debatido sobre el Teatro Arriaga?", en el perfil vectorial, el sistema genera las variantes de la pregunta, busca por los cuatro canales, de los que el literal aporta los fragmentos que contienen el término Arriaga, reordena los candidatos y compone el contexto con los plenos en los que se trató el asunto. El generador redacta un bloque por cada uno y el programa añade los enlaces a las páginas. Ante la pregunta "¿Cuántas proposiciones sobre vivienda ha presentado cada grupo municipal?", en el perfil GraphRAG, el análisis reconoce un recuento por grupo sobre el tema de la vivienda, el programa compone la consulta, el grafo devuelve una fila por grupo y el generador las presenta en una tabla. La consulta que compone el programa para esta pregunta es la siguiente:

```
SELECT ?grupo (COUNT(DISTINCT ?p) AS ?n) WHERE {
  ?p a bo:Proposicion ; bo:fechaISO ?d ; bo:fecha ?fecha .
  FILTER NOT EXISTS { ?p bo:tipoPunto ?_tp .
    FILTER(?_tp IN ("pregunta", "dacion_cuenta", "tramite",
                    "debate_estado_ciudad", "informe_iniciativa")) }
  ?p bo:trataTemaAmplio br:t_vivienda .
  ?p bo:presentadaPor ?g . ?g rdfs:label ?grupo .
  FILTER(?g != br:grupo_desconocido)
} GROUP BY ?grupo ORDER BY DESC(?n) LIMIT 20
```

El primer filtro excluye los puntos que no son proposiciones. La propiedad `bo:trataTemaAmplio` es la que añade el razonador, con lo que la consulta incluye también las proposiciones clasificadas en cualquier subtema de la vivienda. La consulta devuelve ocho filas, encabezadas por EH Bildu, con 51 proposiciones, y el PP, con 44, las mismas de la Figura 9.

Las dos preguntas ilustran, además, la razón de que existan dos sistemas. La primera pide lo que se dijo, que solo figura completo en el texto de las actas. La segunda pide un recuento sobre todo el corpus, que solo puede obtenerse de datos estructurados y que el RAG vectorial no puede resolver, como muestra la Sección 8.5.

6.7. Configuración y decisiones de implementación

6.7.1. Temperatura y parámetros de los LLM

Todos los modelos se utilizan con temperatura 0, para que la respuesta varíe lo menos posible entre ejecuciones. Gemini 3.5 Flash-Lite no permite fijarla, lo que se recoge entre las limitaciones. Se fijan, además, tres parámetros que resultaron necesarios durante el desarrollo:

- **Límite de salida.** El generador local tiene un máximo de 4.096 *tokens* de salida. Sin ese límite, un bucle de repetición mantuvo la aplicación generando durante 50 minutos, como se describe en la Sección 8.3.3.
- **Esfuerzo de razonamiento.** Los modelos gpt-oss-120b y Qwen3 razonan antes de responder, y ese razonamiento consume el límite de salida. Al primero se le solicita un esfuerzo bajo. Al segundo se le desactiva, puesto que no mejora la escritura de una consulta y puede agotar el límite antes de completarla.
- **Respuesta vacía.** En el GraphRAG, una respuesta vacía del proveedor se trata como un fallo y se pasa al proveedor siguiente.

6.7.2. Gestión de cuotas y memoria

Las cuotas de los servicios gratuitos condicionan el diseño tanto como la calidad de los modelos. Groq admite 200.000 *tokens* al día y 8.000 por minuto, y Cohere, 1.000 llamadas al mes. Por ello, cada pieza que depende de un servicio externo dispone de un respaldo que entra de forma automática. Así, el generador pasa de Groq a Gemini y, en último término, al modelo local, y el reordenador, de Cohere al modelo local. El límite del contexto, de 20.000 caracteres, o de 12.000 cuando responde el modelo local, se fija para que una pregunta quepa en una sola petición.

En local, los modelos se ejecutan con Ollama en un equipo con una GPU de 6 GB de memoria, lo que limita su tamaño a unos 8.000 millones de parámetros. El grafo se carga completo en memoria una vez, al arrancar, y los resultados de las operaciones costosas que se repiten, como el texto extraído de cada PDF o el análisis de cada pregunta, se guardan en caché.

6.7.3. Construcción del *prompt* para generación

El *prompt* del RAG vectorial fija el formato de la respuesta con un ejemplo y una lista de reglas, cada una de ellas añadida para corregir un fallo observado. Las principales son

las siguientes: utilizar solo información presente en el contexto; no mezclar el título, el grupo o los votos de una proposición con los de otra; tomar como resultado únicamente el que consta en el bloque, sin confundir la aprobación de una enmienda con la de la proposición; omitir los plenos sin relación con la pregunta, en lugar de escribir un bloque vacío; y advertir de que el contexto es una muestra cuando la pregunta pide un total. El *prompt* del GraphRAG insiste en lo contrario de lo que un modelo haría por defecto: no añadir cifras ausentes de las filas, leer todas las filas antes de señalar un máximo e indicar que no hay datos cuando la consulta no devuelve nada.

Las reglas del *prompt* reducen los fallos, pero no los eliminan, en especial con modelos pequeños. Por ello, los errores más graves se corrigen además en el programa, después de la generación: la eliminación de los bloques repetidos, la colocación de los enlaces a las fuentes y la corrección de los recuentos mal agrupados no dependen de que el modelo obedezca.

7. Corpus y preprocesado de datos

Este capítulo describe los documentos de partida y el proceso que los convierte en los datos con los que trabajan los dos sistemas: los fragmentos de texto del índice vectorial y los puntos del orden del día estructurados del grafo.

7.1. El corpus de actas del Pleno

Como se indicó en la Introducción, de los documentos que el Ayuntamiento de Bilbao publica en su portal web para cada sesión del Pleno, este trabajo utiliza únicamente el acta. La Tabla 34 resume el corpus descargado.

Característica	**Valor**
Actas (una por sesión)	236
Sesiones ordinarias / extraordinarias	135 / 101
Primera y última sesión	12-11-2007 / 30-04-2026
Páginas en total	29.812
Actas escaneadas, sin capa de texto	4

Tabla 34: El corpus de actas del Pleno

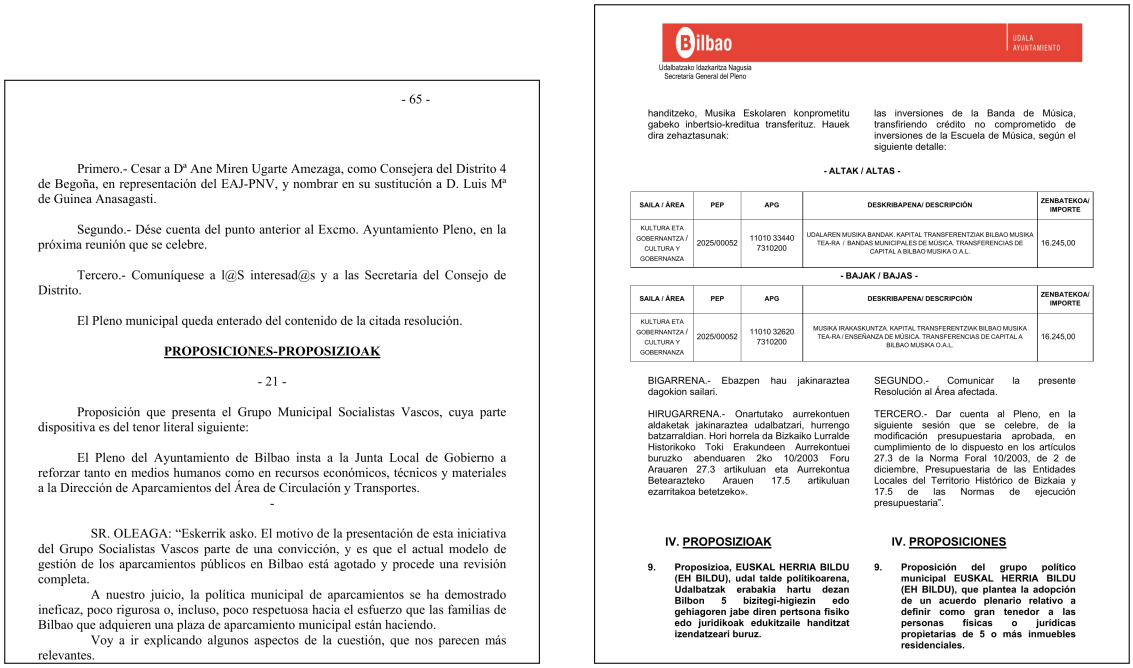
Todas las actas recogen el mismo contenido: la relación de asistentes, los puntos del orden del día y, dentro de cada punto, el texto de la propuesta, las intervenciones de los concejales, cada una precedida del nombre del orador, y el resultado de la votación. Su formato, en cambio, no es uniforme, ya que las actas recientes difieren de forma apreciable de las antiguas. Tres características del corpus condicionan el preprocesado.

En cuanto al formato, hasta 2020, salvo en alguna sesión aislada, las actas están redactadas a una sola columna y en castellano, y numeran los puntos con la forma "- 21 -", que es también la marca que emplean para el número de página. Desde febrero de 2021 se redactan a dos columnas, con el texto en euskera a la izquierda y en castellano a la derecha, y las más recientes agrupan los puntos en apartados y los numeran con la forma "9.". Cambian también la manera de nombrar al grupo que presenta una proposición y la fórmula de la votación. Cuatro actas del periodo 2022–2026 están, además, escaneadas y carecen de capa de texto. La Figura 11 muestra el comienzo de una proposición en un acta de cada formato.

El corpus es, además, bilingüe. Buena parte del contenido aparece dos veces, en euskera y en castellano, sobre todo en las actas a dos columnas, y si esa duplicidad no se detecta, cada proposición se cuenta dos veces.

Por último, no todos los puntos son proposiciones. Las preguntas, las daciones de cuenta, la aprobación del acta anterior o las declaraciones de urgencia son puntos del

orden del día que no se debaten ni se votan como una proposición, y deben distinguirse para no contarlos como tales.



- 65 -

Primero.- Cesar a Dª Ane Miren Ugarte Amezaga, como Consejera del Distrito 4 de Begoña, en representación del EAJ-PNV, y nombrar en su sustitución a D. Luis Mª de Guinea Anasagasti.

Segundo.- Dése cuenta del punto anterior al Excmo. Ayuntamiento Pleno, en la próxima reunión que se celebre.

Tercero.- Comuníquese a l@S interesad@s y a las Secretaria del Consejo de Distrito.

El Pleno municipal queda enterado del contenido de la citada resolución.

PROPOSICIONES-PROPOSIZIOAK

- 21 -

Proposición que presenta el Grupo Municipal Socialistas Vascos, cuya parte dispositiva es del tenor literal siguiente:

El Pleno del Ayuntamiento de Bilbao insta a la Junta Local de Gobierno a reforzar tanto en medios humanos como en recursos económicos, técnicos y materiales a la Dirección de Aparcamientos del Área de Circulación y Transportes.

-

SR. OLEAGA: "Eskerrik asko. El motivo de la presentación de esta iniciativa del Grupo Socialistas Vascos parte de una convicción, y es que el actual modelo de gestión de los aparcamientos públicos en Bilbao está agotado y procede una revisión completa.

A nuestro juicio, la política municipal de aparcamientos se ha demostrado ineficaz, poco rigurosa o, incluso, poco respetuosa hacia el esfuerzo que las familias de Bilbao que adquieren una plaza de aparcamiento municipal están haciendo.

Voy a ir explicando algunos aspectos de la cuestión, que nos parecen más relevantes.

Bilbao

UDALA AYUNTAMIENTO

Udalbatzako Idazkaritza Nagusia
Secretaría General del Pleno

handitzeko, Musika Eskolaren konprometitu gabeko inbertsio-kreditua transferituz. Hauek dira zehaztasunak:

las inversiones de la Banda de Música, transfiriendo crédito no comprometido de inversiones de la Escuela de Música, según el siguiente detalle:

- ALTAK / ALTAS -

SAILA / ÁREA	PEP	APG	DESKRIBAPENA/ DESCRIPCIÓN	ZENBATEKOA/ IMPORTE
KULTURA ETA GOBERNANTZA / CULTURA Y GOBERNANZA	2025/00052	11010 33440 7310200	UDALAREN MUSIKA BANDAK. KAPITAL TRANSFERENTZIAK BILBAO MUSIKA TEA-RA / BANDAS MUNICIPALES DE MÚSICA. TRANSFERENCIAS DE CAPITAL A BILBAO MUSIKA O.A.L.	16.245,00

- BAJAK / BAJAS -

SAILA / ÁREA	PEP	APG	DESKRIBAPENA/ DESCRIPCIÓN	ZENBATEKOA/ IMPORTE
KULTURA ETA GOBERNANTZA / CULTURA Y GOBERNANZA	2025/00052	11010 32620 7310200	MUSIKA IRAKASKUNTZA. KAPITAL TRANSFERENTZIAK BILBAO MUSIKA TEA-RA / ENSEÑANZA DE MÚSICA. TRANSFERENCIAS DE CAPITAL A BILBAO MUSIKA O.A.L.	16.245,00

BIGARRENA.- Ebazpen hau jakinaraztea dagokion sailari.

HIRUGARRENA.- Onartutako aurrekontuen aldaketak jakinaraztea udalbatzari, hurrengo batzarraldian. Hori horrela da Bizkaiko Lurralde Historikoko Toki Erakundeen Aurrekontuei buruzko abenduaren 2ko 10/2003 Foru Arauaren 27.3 artikuluan eta Aurrekontua Betearazteko Arauen 17.5 artikuluan ezarritakoa betetzeko».

SEGUNDO.- Comunicar la presente Resolución al Área afectada.

TERCERO.- Dar cuenta al Pleno, en la siguiente sesión que se celebre, de la modificación presupuestaria aprobada, en cumplimiento de lo dispuesto en los artículos 27.3 de la Norma Foral 10/2003, de 2 de diciembre, Presupuestaria de las Entidades Locales del Territorio Histórico de Bizkaia y 17.5 de las Normas de ejecución presupuestaria".

IV. PROPOSIZIOAK

9. Proposizioa, EUSKAL HERRIA BILDU (EH BILDU), udal talde politikoarena, Udalbatzak erabakia hartu dezan Bilbon 5 bizitegi-higiezin edo gehiagoren jabe diren pertsona fisiko edo juridikoak edukitzaile handitzat izendatzeari buruz.

IV. PROPOSICIONES

9. Proposición del grupo político municipal EUSKAL HERRIA BILDU (EH BILDU), que plantea la adopción de un acuerdo plenario relativo a definir como gran tenedor a las personas físicas o jurídicas propietarias de 5 o más inmuebles residenciales.

Figura 11: Comienzo de una proposición en el acta del 18 de junio de 2008, a la izquierda, y en la del 29 de enero de 2026, a la derecha

Fuente: Ayuntamiento de Bilbao.

7.2. Esquema general del preprocesado

El preprocesado tiene una parte común y dos partes específicas, una por cada estructura de datos, como muestra la Figura 12. La parte común comprende la descarga de las actas, la extracción de su texto y la elaboración del censo de concejales. A partir de ella, el índice vectorial y el grafo se obtienen con procesos distintos. El del índice vectorial es breve y no emplea ningún LLM, pues divide el texto en fragmentos y los convierte en vectores. El del grafo es más largo, porque cada dato debe extraerse del texto antes de almacenarlo. Los dos procesos vuelven a coincidir en un punto, ya que los datos de enriquecimiento que un LLM produce para el grafo se añaden también a los fragmentos del índice vectorial. Así, ambas estructuras disponen de la misma información sobre cada proposición.

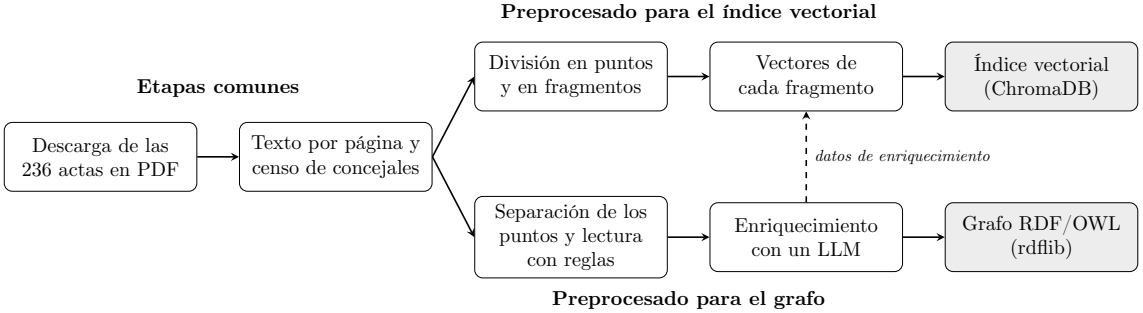


Figura 12: Esquema del preprocesado: etapas comunes y proceso específico de cada estructura de datos

La Tabla 35 detalla los pasos, la estructura de datos a la que sirve cada uno y si emplea un LLM. Los pasos que no lo emplean son deterministas, es decir, producen siempre el

mismo resultado a partir de las mismas actas. Los que sí lo emplean guardan cada resultado en cuanto lo obtienen, lo que permite interrumpirlos y reanudarlos sin repetir el trabajo ya realizado.

Paso	Qué hace	Resultado	Destino	LLM
1	Descarga las actas del portal municipal	Un PDF por sesión	Común	No
2	Extrae el texto de cada página y transcribe las actas escaneadas	Texto de las 236 actas	Común	Solo en 4 actas
3	Elabora el censo de concejales con su grupo	Censo de 93 concejales	Común	No
4	Divide cada acta por puntos y en fragmentos, con sus metadatos	97.217 fragmentos	Índice vectorial	No
5	Convierte cada fragmento en un vector	Índice en ChromaDB	Índice vectorial	No
6	Separa los puntos del orden del día y extrae título, página, grupo y votación	Puntos con sus datos	Grafo	No
7	Separa las intervenciones de cada debate	18.581 intervenciones	Grafo	No
8	Resume y clasifica cada punto: temas, entidades, resultado	Puntos enriquecidos	Grafo e índice vectorial	Sí
9	Resume cada intervención y su postura	Resumen y postura por orador	Grafo	Sí

Tabla 35: Pasos del preprocesado y estructura de datos a la que sirve cada uno

7.3. Etapas comunes

Las tres primeras etapas se realizan una sola vez y sus resultados los utilizan los dos procesos.

7.3.1. Descarga automatizada de las actas

El portal no ofrece una descarga conjunta, ya que es necesario elegir un año en un formulario y abrir cada documento de la tabla de resultados. Un programa propio reproduce ese recorrido mediante *web scraping*, la extracción automatizada de información de las páginas de un sitio web. Para cada año envía el formulario, localiza en la tabla el enlace al acta de cada sesión, descartando el orden del día, los extractos y el resumen, y guarda el fichero en una carpeta por año, con un nombre uniforme formado por la fecha y el tipo de sesión, como `26-01-2017_Ordinaria_Acta.pdf`. El resto del sistema utiliza ese nombre para identificar la sesión de cada dato y para enlazar cada respuesta con su PDF.

7.3.2. Extracción del texto

El texto de cada acta se obtiene página a página y se conserva el número de página de cada pasaje, lo que permite después citar la página exacta, tanto en los fragmentos del índice vectorial como en los puntos del grafo. Las cuatro actas escaneadas se transcribieron con un modelo multimodal, Gemini, también página a página, y su transcripción se trata desde ese momento como el texto de cualquier otra acta.

7.3.3. Censo de concejales

Las actas identifican a quien interviene o vota por su apellido, sin indicar su grupo. Para resolverlo se elabora un censo de los 93 concejales del periodo, con el grupo al que pertenece cada uno y las fechas entre las que asiste a los plenos. Los nombres se obtienen de la relación de asistentes del encabezado de cada acta, y el grupo, de las actas de las sesiones constitutivas de cada mandato, completadas con el sentido del voto en las votaciones nominales. El índice vectorial utiliza este censo para asignar a cada fragmento el grupo del orador. El grafo lo utiliza para resolver el voto nominal y las intervenciones.

7.4. Preprocesado para el índice vectorial

El índice vectorial conserva el texto de las actas, por lo que su preprocesado se limita a dividirlo y a anotar cada fragmento. Cada acta se divide primero por puntos del orden del día, con expresiones regulares, patrones de texto que reconocen el comienzo de un punto en los formatos del corpus, y cada punto, en fragmentos de 1.200 caracteres con 200 de solape. De este modo, ningún fragmento mezcla el final de un debate con el comienzo del siguiente. Es un tamaño intermedio, de unos 300 *tokens*, suficiente para recoger un tramo del debate con su contexto, lo que conviene a las preguntas sobre la posición de un grupo, ya que la literatura sitúa el tamaño óptimo en los fragmentos mayores cuando la pregunta necesita más contexto y en los menores cuando la respuesta es breve y concreta [38].

Cada fragmento conserva como metadatos la fecha del pleno, el título del punto, el orador y su grupo, la página del PDF y el resultado de la votación. El orador se reconoce por la fórmula con la que el acta introduce cada intervención, y su grupo se obtiene del censo de concejales. Al texto del fragmento se le anteponen el título del punto y el orador, para que un fragmento aislado mantenga el contexto de lo que se debate.

Los fragmentos se convierten en vectores con el modelo de *embeddings* nomic-embed-text [39], ejecutado en local, y se almacenan en ChromaDB, con un total de 97.217. A cada uno se le añaden después los datos de su proposición obtenidos en el preprocesado del grafo, descrito en la sección siguiente: el tema, el resultado y el grupo proponente. Con esos mismos datos se construye un segundo índice, de menor tamaño, con un documento breve por cada proposición con resumen, que contiene su título, su grupo, sus temas, sus entidades, su resumen y su resultado. Es el que emplea el canal de proposiciones presentado en la Sección 8.3.2.

7.5. Preprocesado para el grafo

El grafo almacena los hechos que se extraen del texto, por lo que su preprocesado añade la extracción con reglas de los puntos del orden del día con sus datos y su enriquecimiento

posterior con un LLM. Con el resultado se construye después el grafo, como describe la Sección 6.2.

7.5.1. Extracción de los puntos del orden del día

La separación de los puntos del orden del día no emplea ningún LLM. Se realiza con expresiones regulares que reconocen el comienzo de un punto en cada uno de los formatos del corpus. De cada punto se extraen, también mediante reglas, el título y la página en la que comienza. El grupo proponente se obtiene de la fórmula del título, habitualmente del tipo "Proposición que presenta el Grupo Municipal...", con las distintas grafías de cada grupo reducidas a un nombre único. Del texto de la votación se obtienen el resultado, que puede ser aprobada, aprobada con enmienda, rechazada, decae o retirada, y el recuento de votos. El voto nominal de cada concejal, cuando el acta lo recoge, se resuelve con el censo de concejales del mandato correspondiente y de él se deriva el voto de cada grupo. Cuando el acta no lo recoge, el voto de cada grupo procede de la lectura del LLM de la etapa siguiente, lo que ocurre en torno al 7 % de los puntos con voto.

Estos datos se extraen mediante reglas en lugar de con un LLM, porque los votos y el resultado son los datos en los que un error es más grave y la fórmula con la que el acta los recoge es lo bastante regular para leerla sin interpretarla. Cuando el LLM de la etapa siguiente y la regla no coinciden en el resultado, prevalece la regla, conforme al requisito RF3 de la Tabla 1.

La primera extracción produjo 3.422 registros. Las auditorías posteriores con el texto de las actas, descritas en la Sección 8.7, añadieron 1.110 puntos que esa primera pasada no había reconocido, entre ellos iniciativas vecinales, daciones de cuenta y puntos únicos de las sesiones extraordinarias, y eliminaron 692 copias en euskera de puntos ya recogidos en castellano. El resultado son los 3.840 puntos del orden del día que contiene el grafo. Las intervenciones de cada debate se separan por el nombre del orador, también sin LLM, y se asocian al punto en el que se producen.

7.5.2. Enriquecimiento con un LLM

Lo que no puede extraerse con reglas es el significado de cada punto, aquello de lo que trata. Para ello, cada punto se envía a un LLM, que devuelve en un formato estructurado un resumen de lo que se solicita, el tema principal y los subtemas, las entidades mencionadas, que pueden ser empresas, organismos, equipamientos o lugares, y su lectura del resultado y de los votos, que se contrasta con la extraída por reglas. El tema principal no es libre, pues el modelo debe elegirlo de una lista cerrada de 19 temas. Los subtemas, en cambio, son etiquetas libres, que después se hacen corresponder, cuando es posible, con los subtemas de la taxonomía SKOS que comparten los dos sistemas y que se describe en el Capítulo 6. Un segundo paso resume cada intervención y clasifica la postura del orador como favorable, contraria, de abstención o neutra.

El modelo empleado es Gemini 3.5 Flash-Lite, por las razones de coste y de velocidad expuestas en el Capítulo 5. La salida de un LLM no siempre es un dato válido, por lo que cada respuesta se comprueba antes de guardarla. Si no tiene el formato esperado, se intenta reparar y, si no es posible, se repite la petición. Un punto solo se guarda cuando su respuesta es válida.

7.6. Resultado del preprocesado

La Tabla 36 recoge el resultado final del preprocesado, tal como figura en el grafo y en el índice vectorial de la versión entregada.

Elemento	**Cantidad**
Puntos del orden del día	3.840
de ellos, proposiciones	2.702
Proposiciones con voto por grupo	2.122
Votaciones	1.902
Enmiendas	1.640
Intervenciones en los debates	18.581
Concejales / grupos políticos	93 / 14
Entidades mencionadas (organizaciones)	2.059
Triples del grafo	660.727
Fragmentos de texto en el índice vectorial	97.217

Tabla 36: Datos obtenidos del corpus

De los 3.840 puntos del orden del día, 2.702 son proposiciones en sentido estricto. El resto son preguntas, daciones de cuenta, trámites e informes, que se conservan pero no entran en los recuentos de proposiciones.

8. Análisis de resultados

Este capítulo presenta la evaluación de los dos sistemas. Tras el objetivo y la metodología, se exponen los resultados de cada sistema por separado y después su comparación con las mismas preguntas, que es el resultado principal del trabajo. El capítulo termina con los límites de cada sistema, la valoración global y las limitaciones de la evaluación.

8.1. Objetivo de la evaluación

Los dos sistemas son de naturaleza distinta, y esa diferencia condiciona la forma de evaluarlos. El GraphRAG traduce la pregunta a SPARQL y devuelve datos exactos, como recuentos, rankings o proporciones, para los que existe una única respuesta correcta que puede calcularse de antemano. El RAG vectorial responde con una crónica redactada, para la que no existe una única redacción correcta con la que compararla.

La evaluación responde a tres preguntas: si cada sistema cumple aquello para lo que se diseñó, es decir, si el GraphRAG devuelve los datos correctos y si las respuestas del RAG vectorial son pertinentes y se apoyan en las actas recuperadas; cuál de los dos responde mejor a cada tipo de pregunta; y qué se pierde al sustituir los modelos en la nube por alternativas locales.

8.2. Metodología

Se emplearon cinco evaluaciones, que resume la Tabla 37. Las cuatro primeras miden cada sistema por separado, y la quinta los compara con las mismas preguntas.

Evaluación	Sistema	Qué mide	Preguntas
Estudio de generación de SPARQL	GraphRAG	Si la consulta generada devuelve el dato correcto	35
Baterías de regresión	Los dos	Si un cambio altera lo que ya funcionaba	22 y 13
RAGAS	RAG vectorial	Pertinencia del contexto y fidelidad de la respuesta	50
Diagnóstico de la recuperación	RAG vectorial	Si el pleno correcto llega al contexto	102
Preguntas nuevas	Los dos	Aciertos y tipo de fallo ante preguntas no vistas	188

Tabla 37: Evaluaciones realizadas

Además de estas evaluaciones, los dos sistemas se depuraron durante el desarrollo con series de preguntas agrupadas por materia, cuyas respuestas se contrastaban con el texto de las actas. Cuando una respuesta no coincidía con el acta, se corregía la etapa que la había producido, en lugar de la respuesta.

8.2.1. Precisión de ejecución del GraphRAG

El GraphRAG se evalúa por la precisión de ejecución, o *execution accuracy*, el criterio habitual en la literatura sobre traducción de lenguaje natural a consultas formales [17], en el que no se compara el texto redactado, sino las filas que devuelve la consulta con el valor correcto. Es un criterio más riguroso que el de un juez basado en un LLM, ya que compara con un hecho verificado y no con la valoración de otro modelo.

Se aplica en el estudio comparativo de diez modelos sobre 35 preguntas de referencia, descrito en la Sección 5.4, que llevó a seleccionar Qwen3:8B como generador de SPARQL, y en una batería de regresión de 22 casos, con valores esperados y valores prohibidos verificados uno a uno en las actas, que se ejecutó cada vez que cambiaban el grafo o el generador de consultas. El RAG vectorial dispone de una batería análoga, de 13 casos, que verifica la recuperación.

8.2.2. RAGAS para el RAG vectorial

Para el RAG vectorial no existe un único dato correcto con el que comparar, por lo que se recurrió a RAGAS [40], una librería de evaluación de sistemas RAG que emplea un LLM externo como juez (*LLM-as-a-judge*) [41]. Se utilizaron cuatro métricas, todas en el intervalo $[0, 1]$:

- **Context Relevance**: pertinencia temática de los fragmentos recuperados para la pregunta.
- **Faithfulness**: grado en que la respuesta generada se sostiene en el contexto recuperado, sin añadir información ausente de él.
- **Answer Relevancy**: relevancia de la respuesta para la pregunta formulada.
- **Semantic Similarity**: similitud semántica entre la respuesta generada y una respuesta de referencia.

Las tres primeras corresponden a las puntuaciones que la revisión de Gao et al. identifica como núcleo de la evaluación de sistemas RAG, y cubren por separado la recuperación y la generación [5].

La batería consta de 50 preguntas, que combinan preguntas temáticas generales, preguntas acotadas a un grupo político y preguntas sobre un pleno concreto. Se eligieron de forma deliberada temas con pocas proposiciones, para que la respuesta de referencia pudiera verificarse completa. Cada respuesta de referencia se redactó a partir del texto de las actas, y ninguna a partir de lo que el propio sistema recuperaba o respondía.

Como juez se empleó Gemini 2.5 Pro. Un juez local, Qwen3:8B, se descartó por ser más indulgente. En el trabajo de Zheng et al., son los modelos más capaces, como GPT-4, los que alcanzan con los evaluadores humanos un acuerdo comparable al que estos tienen entre sí [41]. El contexto que se entrega al juez es el mismo texto que recibe el generador, incluidos los metadatos de cada fragmento, ya que de lo contrario la fidelidad queda penalizada de forma artificial.

Las respuestas evaluadas son las que produce el sistema tal como funciona en la interfaz. Como generador se empleó Gemini 3.5 Flash-Lite en las 50 preguntas. Aunque el primer generador de la cadena es gpt-oss-120b, la cuota gratuita de Groq hace que en la práctica responda Gemini la mayoría de las veces, como ocurrió en 19 de las 20 respuestas de la última ronda de la Sección 8.2.3. El efecto de cambiar de generador se midió aparte, con el experimento de la Sección 8.3.3.

RAGAS mide la coherencia de la respuesta con el contexto recuperado, pero no si ese contexto o la respuesta son correctos, puesto que no compara con la fuente original. Por ello no sustituye a la verificación con las actas. Por la misma razón no se aplicó a la redacción del GraphRAG, ya que lo que limita a ese sistema es la corrección de la consulta, que ya se mide con la precisión de ejecución, y no la fidelidad de una redacción que parte de datos ya correctos.

8.2.3. Evaluación con preguntas nuevas

Las baterías anteriores comparten una debilidad, y es que sus preguntas se conocían durante el desarrollo y parte de las correcciones se realizaron a partir de ellas. Miden, por tanto, hasta qué punto el sistema se ha ajustado a esas preguntas, pero no su comportamiento ante preguntas imprevistas. Para medir esto último, y para comparar los dos sistemas con las mismas preguntas, se emplearon siete conjuntos de preguntas nuevas, uno por ronda de evaluación, de entre 20 y 40 cada uno, con 188 en total. Las preguntas se generaron con un asistente de IA sin acceso al código y se formularon como lo haría un usuario: recuentos, rankings, votaciones, la última vez que se trató un asunto, lo que se dijo en un debate o preguntas que parten de una premisa falsa.

Tres condiciones hacen que cada ronda sea independiente del desarrollo. El sistema no se modifica desde que se reciben las preguntas hasta que se ejecutan. La respuesta de referencia de cada pregunta y el criterio de corrección se fijan antes de ejecutar. Cada conjunto se pasa una sola vez por los dos sistemas, tal como funcionan en la interfaz, y cuando el sistema se corrige a partir de sus fallos, la ronda siguiente se realiza con un conjunto distinto.

Cada respuesta se califica como correcta, parcial o incorrecta. Las incorrectas se dividen en tres tipos, ya que no tienen la misma gravedad para el usuario: *falsa*, si afirma un dato erróneo sin advertirlo; *honesta*, si reconoce que no puede responder; y *de muestra*, si ofrece una cifra calculada sobre los fragmentos recuperados y advierte de que no es el total. Los resultados se desglosan por tipo de pregunta, porque una cifra global mezcla tipos en los que cada sistema se comporta de forma opuesta.

8.3. Resultados del RAG vectorial

Esta sección presenta las métricas de RAGAS y analiza por separado la recuperación y la generación. La batería de regresión del RAG vectorial se supera completa en la versión entregada, con 13 casos de 13.

8.3.1. Métricas de RAGAS

La Tabla 38 recoge las cuatro métricas sobre las 50 preguntas, con la configuración final del sistema. Las 50 se evaluaron completas, sin ningún fallo del juez.

Métrica	**Media ± desviación típica** ($n = 50$)
Context Relevance	1,00 ± 0,00
Faithfulness	0,98 ± 0,04
Answer Relevancy	0,66 ± 0,09
Semantic Similarity	0,82 ± 0,03

Tabla 38: Métricas RAGAS del RAG vectorial en su configuración final (generador Gemini 3.5 Flash-Lite)

Los fragmentos recuperados son pertinentes en las 50 preguntas, con 1 en Context Relevance. La fidelidad es alta, con una media de 0,98, un valor mínimo de 0,83 y la puntuación máxima en 35 de las 50 preguntas. El sistema apenas afirma nada que no figure en el contexto recuperado.

La métrica más baja es Answer Relevancy, con 0,66. RAGAS la calcula pidiendo al juez preguntas a las que la respuesta contestaría y comparándolas con la original. Las respuestas del sistema son crónicas con un bloque por proposición, que responden a la pregunta pero no con la forma directa que favorece esta métrica, y las de menor puntuación son precisamente los listados de proposiciones. La Semantic Similarity, de 0,82, indica que el contenido coincide en gran parte con la respuesta de referencia.

8.3.2. Recuperación

La recuperación por cuatro canales, presentada en la Sección 5.3, obtiene fragmentos pertinentes en las 50 preguntas de la batería. Para medir la aportación de cada pieza se realizó un diagnóstico sin invocar al generador. Sobre 102 de las preguntas de la Sección 8.2.3, aquellas cuya respuesta de referencia cita un pleno concreto, se comprueba si ese pleno llega al contexto que recibiría el generador. La Tabla 39 recoge el resultado.

Configuración de la recuperación	**Pleno correcto en el contexto** ($n = 102$)
Sin reordenación	34 (33 %)
Reordenador local (bge-m3)	53 (52 %)
Reordenador local + reparto del contexto	66 (65 %)
Cohere + reparto del contexto	72 (71 %)
Cohere + reparto + canal de proposiciones (final)	78 (76 %)

Tabla 39: Diagnóstico de la recuperación: preguntas cuyo pleno correcto llega al contexto del generador

Cada fila de la tabla añade una pieza a la anterior: la reordenación, el reparto del contexto y el canal de proposiciones.

La reordenación depende de un servicio externo, el de Cohere, con una cuota mensual. Durante la evaluación, esa cuota se agotó sin que el sistema lo advirtiera, y los candidatos quedaron sin reordenar. Por ello se añadió un reordenador local de respaldo, que entra de forma automática cuando Cohere falla. Emplea el modelo de *embeddings* multilingüe bge-m3 [42], ejecutado en local, y ordena los candidatos por su similitud coseno con la pregunta. Es un enfoque menos costoso y menos preciso que el de un *cross-encoder* como el de Cohere, que lee de forma conjunta la pregunta y cada fragmento [13]. El reordenador

local recupera buena parte de lo que se pierde sin reordenación, al pasar del 33 % al 52 %, por lo que el sistema puede funcionar sin la API externa con una pérdida moderada.

En cuanto al reparto del contexto, cuando los bloques elegidos superan el límite de caracteres del contexto, ese límite se reparte entre todos, que se recortan a un tamaño común conservando la parte que la búsqueda señaló como relevante. En las versiones iniciales, el contexto se cortaba por el final y, como los bloques se ordenan cronológicamente, se perdían los plenos más recientes y las preguntas sobre la última vez que se trató un asunto se respondían con las actas más antiguas.

El canal de proposiciones busca por significado en el índice de resúmenes descrito en la Sección 7.4 y recupera los fragmentos del debate de las proposiciones que encuentra. Resuelve las preguntas que no emplean las palabras del acta. Así, la expresión "cierre de oficinas bancarias" no comparte vocabulario con "malas prácticas bancarias y cierre de sucursales", pero su significado es el mismo.

8.3.3. Generación

La fidelidad de las respuestas no fue siempre tan alta como muestra la Tabla 38. Durante el desarrollo, con el generador local Qwen2.5-7B, la Faithfulness era baja, de 0,76 de media, muy por debajo de la actual. El análisis de las respuestas peor puntuadas, afirmación por afirmación, mostró tres patrones de error: interpretar "decae" como "aprobada", mezclar información entre bloques e inventar datos ausentes del contexto. El más sistemático es el de "decae", ya que ante el resultado "se aprueba la enmienda del Equipo de Gobierno, por lo que decae la proposición", el generador presenta la proposición como aprobada. Al mezclar información entre bloques, atribuye a un pleno el título, el grupo o los votos de otro. En la clasificación de Ji et al., los dos primeros son alucinaciones intrínsecas, que contradicen el contexto, y el tercero añade información que no figura en él [3]. Los tres son errores de comprensión de un contexto recuperado correctamente, y apuntan al generador y no a la recuperación.

Para comprobarlo se diseñó un experimento controlado. Sobre un subconjunto de 22 preguntas, formado por las 17 con Faithfulness inferior a 0,70 y 5 de control con una puntuación alta, se tomó exactamente el mismo contexto que había recibido el generador local y se redactó de nuevo la respuesta con gpt-oss-120b, con el mismo *prompt*. La única variable que cambia es el modelo que redacta. Las dos versiones se puntuaron con el mismo juez y en la misma pasada. La Tabla 40 recoge el resultado.

Preguntas	**Métrica**	**Qwen2.5-7B**	**gpt-oss-120b**
Peores ($n = 17$)	Faithfulness	0,62 ± 0,16	0,79 ± 0,14
Control ($n = 5$)	Faithfulness	0,70 ± 0,36	0,99 ± 0,03
Todas ($n = 22$)	Faithfulness	0,64 ± 0,22	0,84 ± 0,14
Todas ($n = 22$)	Answer Relevancy	0,62 ± 0,09	0,64 ± 0,10
Todas ($n = 22$)	Semantic Similarity	0,81 ± 0,03	0,82 ± 0,03

Tabla 40: Experimento controlado: mismo contexto recuperado, distinto generador

Con el mismo contexto, gpt-oss-120b obtiene mayor Faithfulness en 20 de las 22 preguntas, empata en una y queda por debajo solo en una. Una prueba de signos sobre esos resultados descarta que la diferencia sea casual ($p < 0{,}001$). La mejora aparece tanto en las preguntas peor puntuadas como en las de control. En estas últimas, la media del generador

local desciende a 0,70 por una sola pregunta, que el juez puntuó con 0 en esta pasada y con el máximo en la anterior, un caso de la variabilidad que se cuantifica en la Sección 8.9. En las cuatro restantes, gpt-oss-120b mejora la puntuación en tres e iguala la otra. Answer Relevancy y Semantic Similarity apenas varían, ya que dependen principalmente de lo recuperado, que era idéntico en las dos condiciones.

La conclusión es que el generador condiciona en gran medida la fidelidad de las respuestas. Es también relevante para decidir cómo desplegar el sistema, ya que un generador local pequeño lo hace independiente de servicios externos, a costa de respuestas menos fieles.

El generador local mostró, además, un fallo más grave. Ante un contexto con varias proposiciones parecidas del mismo día, Qwen2.5-7B entró en un bucle de repetición, un fallo conocido de los LLM con decodificación determinista [43], y continuó generando durante 50 minutos, con más de 52.000 *tokens*. Se fijó un límite de 4.096 *tokens* de salida, que impide que el fallo bloquee la aplicación, pero no evita que el modelo entre en bucle. Es uno de los motivos por los que el generador local queda como último respaldo, detrás de Groq y de Gemini.

8.4. Resultados del GraphRAG

El GraphRAG se evalúa por la corrección de sus consultas. En el estudio de la Sección 5.4, la configuración elegida, con Qwen3:8B, generó una consulta correcta en el 94 % de los casos, y en su repetición con la versión final del grafo y del código, en el 97 %, con 102 consultas correctas de 105. Los tres fallos de la repetición corresponden a preguntas de proporción, el tipo que más falla también en el estudio. La batería de regresión se supera completa en la versión entregada, con 22 casos de 22.

Estas cifras miden la consulta que escribe el LLM, que es solo una de las tres vías del sistema. En la última de las siete rondas de la comparación, 16 de las 20 preguntas se resolvieron con consultas compuestas por el programa, sin intervención de un LLM. Las dos vías se diferencian sobre todo en las respuestas falsas. En la ronda de 40 preguntas, la única desglosada por vía, las consultas compuestas por el programa produjeron 1 respuesta falsa de 22, y las escritas por el LLM, 5 de 17, un 29 %. La pregunta restante recibió una respuesta fija, sin consulta. Las respuestas falsas de esa ronda proceden, por tanto, en su mayoría de las consultas escritas por el LLM, lo que justifica el orden de las tres vías descrito en el Capítulo 6.

8.5. Comparación de los dos sistemas

Los dos sistemas se compararon con los siete conjuntos de preguntas nuevas, 188 preguntas en total. La Figura 13 resume cómo respondió cada uno.

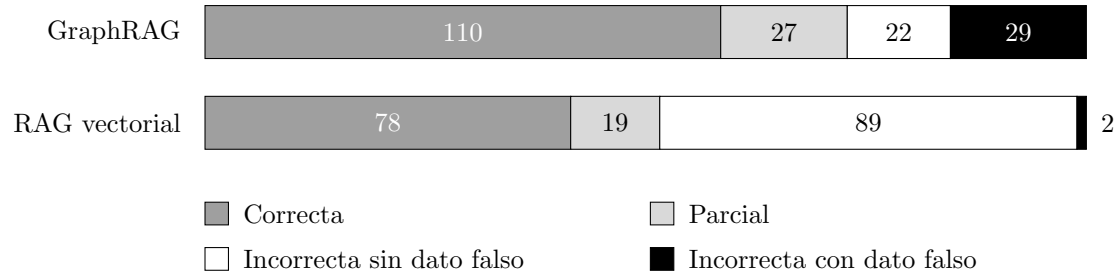


Figura 13: Reparto de las 188 respuestas de cada sistema en las siete rondas con preguntas nuevas

El GraphRAG responde correctamente a más preguntas, 110 frente a 78, pero 29 de sus respuestas afirman un dato erróneo sin advertirlo, frente a 2 del RAG vectorial. Cuando este último no dispone de la respuesta, lo indica o advierte de que trabaja sobre una muestra. La mayor parte de esos casos son recuentos y rankings, que quedan fuera de su alcance.

8.5.1. Resultados por tipo de pregunta

Las 188 preguntas se agrupan en tres tipos: los recuentos y rankings, que piden una cifra o una ordenación sobre el conjunto del corpus; las preguntas sin respuesta en las actas, que parten de una premisa falsa o piden un dato que las actas no recogen; y las preguntas puntuales con respuesta, sobre un asunto, una votación o un debate concretos. La Tabla 41 recoge el resultado de cada sistema en cada tipo.

Tipo de pregunta	n	**GraphRAG**	**RAG vectorial**
Recuentos y rankings	65	34 C / 15 P / 16 I (8 falsas)	2 C / 0 P / 63 I (0 falsas)
Sin respuesta en las actas	52	41 C / 3 P / 8 I (7 falsas)	44 C / 7 P / 1 I (0 falsas)
Puntuales con respuesta	71	35 C / 9 P / 27 I (14 falsas)	32 C / 12 P / 27 I (2 falsas)
Total	**188**	**110 C / 27 P / 51 I (29 falsas)**	**78 C / 19 P / 91 I (2 falsas)**

Tabla 41: Resultados de las siete rondas, por tipo de pregunta. C: correcta; P: parcial; I: incorrecta

Los dos sistemas se diferencian en los recuentos y rankings. El GraphRAG responde correctamente a más de la mitad, y el RAG vectorial, a casi ninguno. La causa está en la estructura del RAG vectorial, que solo dispone de los fragmentos que caben en el contexto, mientras que estas preguntas exigen contar sobre todo el corpus. Fuera de los recuentos y rankings, los dos sistemas obtienen el mismo número de aciertos, 76 de 123 preguntas, de manera que toda la diferencia global procede de ese tipo de pregunta.

En las preguntas sin respuesta en las actas, los dos sistemas responden bien en la mayoría de los casos, al indicar que el dato no consta. La diferencia está en los fallos, pues el GraphRAG afirma en ocasiones un dato y el RAG vectorial, nunca.

En las preguntas puntuales, los dos sistemas aciertan en una proporción similar y se diferencian de nuevo en la forma de fallar. La mitad de las respuestas incorrectas del

GraphRAG afirman un dato erróneo. Las del RAG vectorial apenas lo hacen, porque el sistema indica que no dispone del dato.

La Tabla 42 recoge el mismo desglose para la última ronda, de 20 preguntas.

Tipo de pregunta	n	**GraphRAG**	**RAG vectorial**
Recuentos y rankings	7	6 C / 0 P / 1 I (1 falsa)	0 C / 0 P / 7 I (6 de muestra)
Sin respuesta en las actas	6	6 C	5 C / 1 P
Puntuales con respuesta	7	2 C / 2 P / 3 I (3 honestas)	3 C / 4 P / 0 I
Total	20	**14 C / 2 P / 4 I**	**8 C / 5 P / 7 I**

Tabla 42: Última ronda (20 preguntas), por tipo de pregunta

En los recuentos y en las preguntas sin respuesta se repite el patrón del conjunto. En las puntuales, el RAG vectorial no falla ninguna, con 3 correctas y 4 parciales, y el GraphRAG falla tres, de forma honesta, al responder que el dato no consta. La Tabla 43 reúne un ejemplo de cada caso.

Pregunta	**GraphRAG**	**RAG vectorial**
¿Cuántas proposiciones del PP se rechazaron en la legislatura 2019-2023?	Correcta: 15	Incorrecta, de muestra: encuentra una y advierte de que la cifra procede de los fragmentos recuperados
¿Qué grupo ha presentado más proposiciones sobre pobreza y exclusión social?	Incorrecta, falsa: da 17 a un grupo al contar todo el tema de exclusión social; la referencia es EH Bildu, con 3	Incorrecta, de muestra: fragmentos de un solo año, sin ranking
¿Se ha hablado en el Pleno de los kioscos de prensa?	Incorrecta, honesta: responde que no consta, porque el acta escribe “quioscos”	Correcta: cita la proposición del 27 de junio de 2024
¿Por qué rechazó el Pleno por unanimidad la proposición del PSE-EE para vender la Alhóndiga?	Correcta: no consta esa proposición	Correcta: la premisa no aparece en las actas

Tabla 43: Ejemplos de respuestas de la última ronda

8.5.2. Causas de los fallos de cada sistema

Los dos sistemas fallan, además, por razones distintas.

Los fallos del GraphRAG tienen dos causas, que a menudo se combinan. Una es la formulación de la pregunta, que admite muchas redacciones, mientras que el sistema identifica el asunto por las palabras del título de las proposiciones y por su clasificación

temática. Cuando la pregunta emplea otras palabras u otra grafía, la consulta que se construye busca algo distinto de lo que debería. En unos casos no encuentra nada y el sistema responde que el dato no consta, y en otros encuentra un asunto parecido y lo presenta como respuesta. Más de la mitad de sus respuestas incorrectas tienen este origen. La otra causa es la consulta escrita por el LLM. Cuando la pregunta no corresponde a ninguno de los tipos previstos, un LLM escribe la consulta, y en ocasiones omite un filtro, aplica un criterio que no corresponde a la pregunta o da por buena una premisa falsa. La consulta se ejecuta sin error y devuelve filas, por lo que el fallo no se advierte. De esta vía procede la mayor parte de las respuestas falsas en la ronda desglosada por vía, como muestra la Sección 8.4.

Los fallos del RAG vectorial tienen una causa principal, la recuperación. La respuesta está en las actas, pero el fragmento que la contiene no llega al contexto. Este fallo depende de la reordenación y de la composición del contexto. Con la configuración final, este problema se reduce de forma notable. En las rondas en las que el sistema funcionó sin reordenación falló cerca de la mitad de las preguntas puntuales, mientras que, según el diagnóstico de recuperación, con la configuración final el pleno correcto llega al contexto en el 85 % de las 52 preguntas puntuales del diagnóstico, frente al 38 % sin reordenación. Sobre el total de las 102 preguntas del diagnóstico la proporción es menor, del 76 % (Tabla 39), porque estas incluyen también preguntas con premisa falsa, preguntas ambiguas y algunos recuentos. Sus escasas respuestas falsas corresponden a preguntas sobre la última vez que se trató un asunto, en las que toma como más reciente una fecha en la que el tema solo se menciona de pasada. No se cuentan aquí como fallos los recuentos y rankings, que el sistema no puede resolver por su propia estructura y que se tratan en la Sección 8.6.

La diferencia en la forma de fallar se explica por el modo en que responde cada sistema. El RAG vectorial redacta a partir de fragmentos de texto, y cuando esos fragmentos no contienen la respuesta, el propio texto lo muestra y el sistema lo indica. El GraphRAG redacta a partir de las filas que devuelve una consulta, y unas filas que no corresponden a lo preguntado tienen el mismo aspecto que las correctas.

8.5.3. Evolución a lo largo de las siete rondas

Con 20 preguntas, cada respuesta modifica el resultado en cinco puntos, y una ronda no es, por tanto, suficiente por sí sola. La Tabla 44 reúne las siete, de R1 a R7. Cada una corresponde a una versión distinta del sistema, y la tabla describe una trayectoria, no repeticiones de la misma ronda.

		GraphRAG		RAG vectorial	
Ronda	n	**Correctas**	**Falsas**	**Correctas**	**Falsas**
R1	20	13	3 (15 %)	15	0
R2	40	23	6 (15 %)	17	0
R3	30	20	3 (10 %)	19	0
R4	25	14	4 (16 %)	8	1 (4 %)
R5	24	12	4 (17 %)	5	0
R6	29	14	8 (28 %)	6	1 (3 %)
R7	20	14	1 (5 %)	8	0
Total	**188**	**110 (59 %)**	**29 (15 %)**	**78 (41 %)**	**2 (1 %)**

Tabla 44: Rondas con preguntas nuevas a lo largo del desarrollo. R7 es la de la Tabla 42. En R2–R6 el RAG vectorial funcionó sin reordenación (cuota de Cohere agotada sin que el sistema lo advirtiera)

Si se analizan las siete rondas en conjunto, se pueden deducir tres conclusiones. El GraphRAG responde correctamente a más preguntas, un 59 % frente a un 41 %, principalmente porque resuelve los recuentos y rankings que el RAG vectorial no puede resolver. Ocurre en todas las rondas salvo en R1, cuyo conjunto solo incluía cuatro recuentos o rankings y en la que el RAG vectorial obtuvo 15 respuestas correctas frente a 13. La cifra global depende, por tanto, de la proporción de cada tipo de pregunta en los conjuntos, y está además condicionada por las cinco rondas en las que el RAG vectorial funcionó sin reordenación. El RAG vectorial, en cambio, solo afirmó un dato falso en 2 de las 188 preguntas. Por último, el GraphRAG, cuando falla, lo hace a menudo sin advertirlo, pues una de cada siete respuestas, el 15 %, afirma un dato erróneo, y la proporción oscila entre el 5 % y el 28 % según la ronda.

Las dos proporciones más altas de respuestas falsas, las de R5 y R6, coinciden con las dos ampliaciones del grafo, que añadieron más de mil puntos del orden del día, las intervenciones de los debates y nuevos tipos de dato. Cada dato nuevo introduce nuevas posibilidades de error, analizadas en la Sección 8.7, que hubo que corregir antes de R7.

8.5.4. Adecuación de cada sistema al tipo de pregunta

Los resultados delimitan para qué conviene cada sistema. El GraphRAG es la opción adecuada para las preguntas analíticas sobre el corpus completo, como los recuentos, los rankings y las comparaciones entre grupos o a lo largo de los años, ya que puede agregar miles de proposiciones con una única consulta y devolver una cifra exacta y verificable. Es la misma limitación de la recuperación vectorial que motiva el trabajo de Edge et al., según el cual la recuperación de fragmentos aislados no permite responder a preguntas globales dirigidas al corpus completo [6]. Su propuesta la resuelve con resúmenes de comunidades del grafo generados por un LLM, mientras que este trabajo lo hace con consultas SPARQL sobre datos estructurados, pero el diagnóstico de partida es el mismo. En las preguntas puntuales, en cambio, ninguno de los dos acierta más que el otro de forma apreciable, y lo que distingue al RAG vectorial es su fiabilidad, dado que apenas da respuestas falsas y lo que responde se apoya en el texto del acta, que puede citar y resumir. El GraphRAG opera sobre datos estructurados y resúmenes, y no dispone de las palabras del debate.

Para un usuario no es equivalente un sistema que responde a menos preguntas pero no da datos falsos que otro que responde a más pero en ocasiones se equivoca sin advertirlo,

y esa diferencia es tan relevante como el porcentaje de aciertos a la hora de elegir entre ambos.

8.6. Límites del RAG vectorial

El RAG vectorial tiene un límite que no depende de su ajuste, sino de su estructura, y es que no puede responder a las preguntas que exigen contar u ordenar sobre el conjunto del corpus. El sistema responde a partir de los fragmentos que caben en el contexto, que proceden de unas pocas sesiones, y un recuento o un ranking exige recorrer las 236 actas. Ninguna mejora de la recuperación o del generador lo resuelve. Por ello, estas preguntas no deben considerarse fallos del sistema, sino preguntas ajenas a su alcance, y son precisamente las que justifican la construcción del grafo. El sistema, además, lo advierte, ya que cuando se le pide una cifra indica que la calcula sobre una muestra y no sobre el total.

Su segundo límite es la dependencia de la recuperación. La respuesta solo puede ser correcta si el fragmento que la contiene llega al contexto, y eso depende principalmente de la reordenación, que en la configuración final corresponde a un servicio externo. Sin reordenación, el pleno correcto llega al contexto en una de cada tres preguntas, y con la configuración completa, en tres de cada cuatro, como recoge la Tabla 39.

8.7. Límites del Knowledge Graph

El planteamiento del trabajo es la comparación de dos estructuras de datos, por lo que durante el desarrollo se procuró que el grafo contuviera la misma información que el índice vectorial, dado que si al grafo le falta un dato, la diferencia de resultados no se debe a la estructura, sino a una carencia. Con ese criterio, el grafo se auditó con el texto de las actas y se completó en varias ocasiones. Aun así, existen razones de fondo por las que el grafo no puede contener todo lo que recogen las actas ni responder a cualquier pregunta. No son fallos de esta implementación, sino consecuencias del enfoque.

8.7.1. Información limitada a lo previsto en la ontología

Un índice vectorial almacena el texto tal como es y decide qué es relevante en el momento de la pregunta. Un grafo obliga a decidir de antemano qué se almacena y con qué forma. Lo que no encaja en sus clases se pierde o se reduce. El ejemplo más claro son los debates, cuyas 18.581 intervenciones figuran en el grafo resumidas, con la postura de cada orador, por los motivos expuestos en la Sección 6.2. El grafo contiene que un grupo intervino en contra de una proposición y un resumen de sus motivos, pero no las palabras exactas que empleó. Por ello falla cuando la pregunta busca una palabra que solo figura en el texto del acta.

8.7.2. Errores de la extracción automática

El grafo no se escribe manualmente, sino que se obtiene de las actas con los dos procedimientos descritos en el Capítulo 7: las reglas, que separan los puntos y leen el resultado y los votos, y un LLM, que resume cada punto y extrae sus temas y sus entidades. Ninguno de los dos está libre de error. Las reglas fallan ante los formatos y las fórmulas que no prevén, y el LLM no es determinista y se equivoca con una frecuencia reducida pero

no nula. Con miles de puntos, una frecuencia reducida supone decenas o cientos de datos erróneos. La auditoría del grafo con las actas encontró, entre otros, los errores siguientes:

- 82 registros con un resultado que no concordaba con el texto de su votación, la mayoría por no reconocerse la fórmula "decae la proposición", reducidos a 25 al incorporarla;
- cerca de 700 proposiciones duplicadas por figurar en castellano y en euskera, que alteraban los rankings;
- más de mil puntos del orden del día sin extraer, entre ellos iniciativas vecinales, daciones de cuenta y puntos únicos de las sesiones extraordinarias, y once actas de 2021 y 2022 con entre 0 y 4 proposiciones de las 6 a 10 que contenían;
- 17 proposiciones duplicadas y 66 resultados tomados de la votación del punto contiguo.

Los errores detectados se corrigieron, pero la conclusión es general. Cuanta más información se incorpora al grafo, más puntos existen en los que la extracción puede clasificar mal, omitir o duplicar, y cada tipo de dato nuevo exige su propia auditoría con el original. En el índice vectorial este problema apenas existe, ya que el texto del acta no se interpreta al almacenarlo, solo se divide, y un error de interpretación del generador afecta a una respuesta, y no a todas las que dependen de ese dato.

8.7.3. Dependencia de la corrección de la consulta

Para responder, el GraphRAG debe convertir la pregunta en una consulta SPARQL. Las preguntas de los tipos previstos se resuelven con consultas compuestas por el programa, y las demás, con una consulta escrita por un LLM, que es la vía menos fiable, como muestra la Sección 8.4. El grafo responde bien, por tanto, a lo que se supo anticipar en su diseño, y cada tipo de pregunta nuevo exige programar y mantener su consulta.

Lo imprevisto queda a cargo de un LLM, cuyos errores son los más peligrosos del sistema, porque una consulta errónea se ejecuta sin error y devuelve filas. Así, ante la pregunta "¿cuántas proposiciones han decaído en 2019?", un modelo generó una consulta que contaba todas las proposiciones de 2019, que eran 111, sin filtrar por resultado. El resultado fue una cifra exacta, verificable en apariencia y falsa. El RAG vectorial, que no traduce la pregunta a ningún lenguaje formal, carece de este punto de fallo, y su límite es el opuesto, la imposibilidad de agregar sobre todo el corpus.

Una alternativa que atenúa los tres límites consiste en conectar el grafo con el texto, de forma que el grafo localice las proposiciones que responden a la pregunta y el texto de sus actas aporte lo que se dijo. Se desarrolló un prototipo durante el proyecto y se desactivó para comparar las dos estructuras en igualdad de condiciones. Se recoge como trabajo futuro en la Sección 9.5.

8.8. Valoración global

Los resultados responden a las tres preguntas planteadas al comienzo del capítulo.

Cada sistema cumple aquello para lo que se diseñó. El RAG vectorial recupera contexto pertinente en todas las preguntas de la batería, con una Context Relevance de 1,00, y redacta respuestas fieles a él, con una Faithfulness de 0,98. El GraphRAG genera una

consulta correcta en el 97 % de los casos de la repetición del estudio y supera por completo su batería de regresión.

Ninguno de los dos es mejor para todas las preguntas. El GraphRAG responde correctamente a más preguntas, un 59 % frente a un 41 %, porque resuelve los recuentos y rankings sobre el corpus completo, que quedan fuera del alcance del RAG vectorial, a costa de que una de cada siete respuestas afirme un dato erróneo. En las preguntas puntuales, los dos aciertan en una proporción similar, y el RAG vectorial es el sistema que menos datos falsos ofrece, con 2 respuestas falsas en 188 preguntas. Los fallos del GraphRAG proceden de la formulación de la pregunta y de las consultas escritas por el LLM, y los del RAG vectorial, de que el fragmento pertinente no llegue al contexto.

Las alternativas locales hacen viable el sistema sin depender de servicios externos, pero con peores resultados. El reordenador local sitúa el pleno correcto en el contexto en el 65 % de las preguntas, frente al 71 % de Cohere, y el generador local es menos fiel al contexto que los modelos en la nube.

8.9. Limitaciones

La evaluación tiene las limitaciones siguientes.

- **Variabilidad del juez.** Un juez basado en un LLM no es determinista, una fuente de ruido conocida de esta metodología [44]. Al puntuar por segunda vez las mismas 22 respuestas del generador local, la Faithfulness de una misma pregunta varió de media 0,16, y hasta 1,00 en un caso extremo, mientras que la media del conjunto apenas cambió, en 0,003. Por ello, los resultados se ofrecen como media y desviación típica sobre toda la batería, y la puntuación de una pregunta aislada no debe interpretarse por sí sola.
- **Generador de la evaluación.** Las métricas de la Tabla 38 corresponden a Gemini 3.5 Flash-Lite, el generador que responde la mayoría de las veces. Con gpt-oss-120b solo se midió el subconjunto de 22 preguntas del experimento controlado, dado que la cuota gratuita de Groq no permite redactar las 50 en una jornada. Además, Gemini 3.5 no admite fijar la temperatura a 0, de manera que la misma pregunta puede redactarse de forma ligeramente distinta en dos ejecuciones.
- **Recuperación no determinista.** Dos ejecuciones del mismo código pueden entregar a la reordenación listas de candidatos ligeramente distintas, como se comprobó en al menos la mitad de las preguntas de una muestra. La causa más probable es que los *embeddings* calculados en la GPU no son idénticos de una llamada a otra.
- **Tamaño de los conjuntos de preguntas.** Cada conjunto tiene entre 20 y 40 preguntas, con lo que una respuesta modifica el resultado en varios puntos, y cada uno corresponde a una versión distinta del sistema. En cinco de los siete, el RAG vectorial funcionó sin reordenación. La calificación se realizó respuesta a respuesta, con referencias fijadas antes de ejecutar, pero sin un segundo calificador que permita medir el acuerdo entre ambos.
- **Versión evaluada.** Las siete rondas de la comparación son anteriores a la última revisión del código, que corrigió defectos menores y reconstruyó el grafo y el índice. No se repitieron, ya que cada conjunto requiere calificación manual. Como comprobación, se pasaron de nuevo por el GraphRAG diez preguntas de la última ronda, y en nueve la consulta y las filas devueltas fueron idénticas. La décima dependía de una consulta escrita por el LLM y se resolvió con una consulta compuesta

por el programa. La evaluación con RAGAS y las baterías de regresión sí se repitieron completas tras la revisión.

- **Batería de referencia.** Las 50 preguntas bastan para que las medias sean estables frente al ruido del juez, como muestra la variación de 0,003 en la media al puntuar de nuevo, pero no cubren todos los tipos de consulta posibles. La elección de temas con pocas proposiciones, necesaria para verificar completas las respuestas de referencia, deja fuera los temas más amplios del corpus.

9. Conclusiones y trabajo futuro

Este capítulo recoge las conclusiones del trabajo, el grado de cumplimiento de los objetivos, las conclusiones sobre las pruebas realizadas, las contribuciones y las líneas de trabajo futuro.

9.1. Conclusiones del trabajo

Este trabajo ha construido dos sistemas que responden en lenguaje natural a preguntas sobre las actas del Pleno del Ayuntamiento de Bilbao del periodo 2007–2026. Los dos parten de los mismos documentos y de la misma información, organizada en dos estructuras de datos distintas: un índice vectorial de fragmentos de texto y un Knowledge Graph RDF consultado con SPARQL. La cuestión de fondo era cuál de las dos estructuras es más adecuada para un corpus de este tipo. Los resultados obtenidos permiten tres conclusiones, sobre la estructura más adecuada, la limitación del grafo y los modelos empleados.

Para un corpus como este, el grafo es la estructura más adecuada. Es la única que resuelve las preguntas de agregación sobre el corpus completo, como los recuentos, los rankings o la evolución por años, que el índice vectorial no puede abordar por su propia estructura, ya que solo dispone de los fragmentos que caben en su contexto. En el resto de las preguntas, los dos sistemas acertaron en la misma proporción. Su punto débil es la fiabilidad. Al no contener el texto de los debates, el grafo no capta todos los matices de la pregunta ni de lo que se dijo, y en ocasiones presenta como respuesta una proposición que solo se parece a la pedida. Ese es el problema que aborda el grafo conectado al texto que se propone como trabajo futuro.

La otra limitación del grafo está en cómo se le pregunta. Para responder, el sistema debe convertir la pregunta en una consulta, y esa conversión falla por dos motivos: la ambigüedad con la que el usuario formula la pregunta, que puede emplear palabras distintas de las del grafo, y los errores del LLM al escribir la consulta. El fallo, además, no se advierte, porque una consulta que no corresponde a la pregunta se ejecuta sin error y devuelve filas con el mismo aspecto que las correctas. Según los resultados obtenidos, una de cada siete respuestas del GraphRAG afirmó un dato erróneo. El RAG vectorial, en cambio, apenas afirma datos falsos, pues cuando el fragmento pertinente no llega al contexto, lo indica. Su fallo es de recuperación, y se reduce de forma notable con la configuración final. La fiabilidad es, por tanto, la ventaja del índice vectorial y el principal problema pendiente del grafo.

En cuanto a los modelos, las alternativas locales al generador, al reordenador y al modelo que escribe las consultas hacen viable el sistema sin depender de servicios externos y se mantienen como respaldo automático. Sin embargo, en las tres piezas evaluadas los mejores resultados se obtienen con los modelos más potentes. El generador en la nube es más fiel al contexto que el local, Cohere ordena mejor los candidatos que el reordenador local y, en la generación de SPARQL, el mejor modelo del estudio fue también uno en la nube, aunque un modelo local de 8.000 millones de parámetros con un *prompt* adecuado

quedó cerca. La dependencia de las API externas puede reducirse, pero a costa de la calidad.

Lo que el grafo de este trabajo no contiene es el texto íntegro de los debates. Las intervenciones figuran como un resumen de la postura de cada orador, sin sus palabras literales. El texto completo no se incorporó porque habría duplicado el índice vectorial en lugar de ofrecer una estructura distinta con la que compararlo, y porque habría más que duplicado el tamaño del grafo, como se expone en la Sección 6.2. Un grafo enlazado con el texto de las actas, en el que el grafo localice las proposiciones y el texto aporte lo que se dijo, podría responder con un único sistema a los dos tipos de pregunta, y es la principal línea de continuación de este trabajo. El texto le permitiría captar los matices que hoy se pierden, como las menciones de un asunto fuera del título de la proposición o las palabras con las que se formula la pregunta, aunque seguiría dependiendo de una consulta bien construida en las preguntas ambiguas.

La preferencia por el grafo no es general, sino propia de este tipo de corpus. El RAG vectorial es el enfoque más extendido, y en su uso más habitual, como un asistente sobre la documentación interna de una organización, las preguntas son puntuales, la respuesta se encuentra en uno o en pocos pasajes y rara vez se pide contar o comparar sobre todo el corpus. En ese caso, un RAG vectorial bien construido es suficiente, y construir un grafo supondría un coste elevado para una mejora reducida, puesto que exige diseñar una ontología, extraer cada dato, validar el resultado y auditarlo con el original, mientras que el índice vectorial solo exige indexar los documentos. Las actas de un pleno representan el caso contrario. Las preguntas de recuento y de ranking, como qué grupo ha presentado más proposiciones sobre vivienda o cuántas se aprobaron en un año, son naturales y frecuentes en el uso que harían periodistas, investigadores o ciudadanos, y el dominio tiene una estructura regular, formada por grupos, proposiciones, votaciones, fechas y concejales, que se ajusta bien a un grafo. En este dominio, el coste de construir el grafo está justificado, porque es la única forma de responder a una parte importante de las preguntas.

9.2. Cumplimiento de objetivos

El objetivo general del trabajo era desarrollar un sistema que permitiera consultar en lenguaje natural las actas del Pleno y que respondiera con datos verificables, enlazados a la página del documento de origen, y se ha cumplido, con la excepción de los resultados agregados del GraphRAG, como un recuento. Estos se calculan sobre todo el corpus y no proceden de un documento ni de una página concreta, por lo que no pueden llevar cita. Los seis objetivos específicos de la Sección 1.2.2 se han cumplido también, el cuarto con una excepción:

1. **Extracción automatizada de las proposiciones.** Cumplido: de las 236 actas se han extraído 3.840 puntos del orden del día con su fecha, el grupo que los presenta, el tema, las entidades, el resultado y la página, como describe el Capítulo 7.
2. **Knowledge Graph RDF/OWL con razonamiento.** Cumplido: el grafo, de unos 660.000 triples, se valida con SHACL sin violaciones y deriva con un razonador la herencia entre temas, como describe el Capítulo 6.
3. **RAG vectorial con recuperación híbrida.** Cumplido: la recuperación por cuatro canales obtiene contexto pertinente en las 50 preguntas de la batería, según la Tabla 38.
4. **GraphRAG para preguntas de agregación.** Cumplido en su propósito: resuelve los recuentos y rankings que el RAG vectorial no puede resolver. La excepción es su

fiabilidad, ya que una de cada siete respuestas afirma un dato erróneo, por las causas que analiza la Sección 8.5.2.

5. **Interfaz conversacional con cita de la página.** Cumplido: los dos sistemas se consultan desde la misma interfaz, y cada respuesta enlaza con el PDF de origen y, cuando procede de una proposición o de un fragmento concretos, con su página.
6. **Evaluación comparativa.** Cumplido y ampliado respecto a lo previsto: además de la batería de referencia, los dos sistemas se compararon con preguntas no empleadas en su desarrollo, como recoge la Sección 8.5.

No se ha realizado una evaluación con usuarios reales, que se recoge como trabajo futuro en la Sección 9.5.

9.3. Conclusiones sobre las pruebas realizadas

Las evaluaciones del Capítulo 8 miden aspectos distintos y se complementan. Las baterías de regresión, con resultados verificados en las actas, permitieron comprobar que cada cambio no alteraba lo que ya funcionaba. RAGAS muestra que el RAG vectorial recupera contexto pertinente en todas las preguntas de la batería y que sus respuestas son fieles a él, y el experimento con el contexto fijo, que esa fidelidad depende en gran medida del modelo que redacta. El diagnóstico de la recuperación cuantificó la aportación de cada pieza y mostró que la reordenación es la que más influye. La comparación con preguntas no empleadas en el desarrollo, por último, es la única que mide el comportamiento de los sistemas ante preguntas imprevistas y la única que los compara entre sí, por lo que es la que sostiene las conclusiones.

Las pruebas dejan dos enseñanzas metodológicas. Una cifra global oculta lo que se pretende comparar. El porcentaje de aciertos de cada sistema depende de cuántos recuentos contenga el conjunto de preguntas, y solo el desglose por tipo de pregunta muestra en qué se diferencian. Además, varias de las mejoras más importantes surgieron del examen de los fallos uno a uno con las actas, más que de una métrica. Es el caso de la fórmula "decae" mal clasificada, del recorte de los bloques del contexto y de los puntos ausentes del grafo. Una métrica alta puede coexistir con un fallo sistemático si la batería no lo pone a prueba.

Las pruebas tienen también límites, recogidos en la Sección 8.9. Los conjuntos de preguntas empleados en la comparación son pequeños, corresponden a versiones sucesivas del sistema y su calificación no se contrastó con un segundo calificador.

9.4. Contribuciones científico-técnicas

Las aportaciones de este trabajo son las siguientes:

- **Un corpus estructurado de la actividad del Pleno de Bilbao entre 2007 y 2026**: un grafo RDF con ontología propia que recoge proposiciones, votaciones, enmiendas, intervenciones, grupos y concejales, con la página del acta de la que procede cada punto. Al ser RDF estándar, puede consultarse y enlazarse con otras fuentes con independencia del sistema construido.
- **Una comparación de dos estructuras de datos sobre los mismos documentos**, en la que se ha procurado que el grafo y el índice vectorial partan de la misma información, para que la diferencia de resultados pueda atribuirse a la estructura y no al contenido de cada una.

- **Un método de evaluación con preguntas nuevas**, descrito en la Sección 8.2.3, que mantiene el sistema sin modificar desde que se reciben las preguntas, fija las referencias antes de ejecutar y distingue, entre las respuestas incorrectas, la que afirma un dato falso de la que reconoce no disponer de él.
- **Tres resultados** de utilidad para quien construya un sistema similar: fuera de las preguntas de agregación, las dos estructuras aciertan lo mismo, y lo que las distingue es la forma de fallar; las consultas compuestas por el programa dan una respuesta falsa con la mitad de frecuencia que las escritas por un LLM, un 13 % frente a un 25 %, con un porcentaje de aciertos similar; y, con el mismo contexto recuperado, cambiar el modelo que redacta eleva la fidelidad de las respuestas de 0,64 a 0,84.
- **El código y los datos del proyecto**, disponibles en un repositorio público: el código de los dos sistemas, la ontología, el grafo, las baterías de regresión y la batería de RAGAS con sus resultados (`https://github.com/plopez053/TFG`).

9.5. Trabajo futuro

Las principales líneas de continuación son las siguientes:

- **Grafo conectado al texto.** Es la línea principal, como se ha expuesto en las conclusiones. Consiste en que cada proposición y cada intervención del grafo apunten a los fragmentos de las actas de los que proceden. Así, el grafo identificaría qué responde a la pregunta, con la exactitud de una consulta, y el texto aportaría lo que se dijo, con la riqueza del original. A diferencia de las dos opciones descartadas en la Sección 6.2, no duplica el texto en el grafo ni exige que un LLM extraiga más datos, con lo que no añade nuevas fuentes de error. Atenuaría la pérdida de información que supone resumir los debates y el fallo de las preguntas que emplean palabras distintas de las del grafo, que está en el origen de buena parte de sus respuestas erróneas. Durante el proyecto se desarrolló un prototipo, que se desactivó para comparar las dos estructuras en igualdad de condiciones. Es también la idea de fondo del método GraphRAG de Edge et al., que conserva enlazados las entidades del grafo y los fragmentos de texto de los que se extrajeron [6].
- **Más consultas compuestas por el programa.** Ampliar los tipos de pregunta que se resuelven sin que un LLM escriba la consulta, y avisar cuando el asunto encontrado no coincide con el preguntado, es la forma más directa de reducir las respuestas falsas del GraphRAG.
- **Enrutado automático entre los dos sistemas.** Mientras los dos sistemas se mantengan separados, un enrutador puede decidir, para cada pregunta, si la responde el grafo o el índice vectorial. Se probó uno basado en reglas, que enviaba al grafo los recuentos, los rankings y las votaciones, y al texto las preguntas sobre lo que se dijo y sobre la última vez que se trató un asunto. No mejoró los resultados del GraphRAG por sí solo, y no se incluyó, por ello, en la versión final. Un enrutador mejor requeriría una clasificación más fina de la intención de la pregunta y la posibilidad de recurrir al otro sistema cuando el elegido no encuentra nada. Trabajos como HybridRAG [45] van más allá y combinan en un mismo contexto lo recuperado del grafo y del índice vectorial, con mejores resultados que cada uno por separado.
- **Reordenación sin dependencia externa.** La recuperación del RAG vectorial depende de un servicio externo de reordenación, y el reordenador local de respaldo es menos preciso. Un modelo de reordenación de tipo *cross-encoder* ejecutado en local reduciría esa dependencia, y queda por medir con qué pérdida de precisión.

- **Evaluación más amplia.** La comparación ganaría solidez con conjuntos de preguntas mayores, con un segundo calificador que permita medir el acuerdo y con usuarios reales, como periodistas o investigadores, que formulen sus propias preguntas.
- **Enlace con datos abiertos enlazados.** Al ser RDF estándar, el grafo puede enlazarse con fuentes externas: los temas, con el tesauro EuroVoc [46] de la Unión Europea, concebido para documentos del sector público, y las entidades, como empresas, equipamientos o barrios, con Wikidata [47], lo que aportaría, por ejemplo, las coordenadas necesarias para situar las proposiciones en un mapa. Es una ventaja de la estructura de grafo que el índice vectorial no ofrece, ya que sus fragmentos son texto sin identidad propia. Durante el proyecto se valoró ese enlace para completar los datos de los concejales y se descartó, ya que en torno a tres de cada cuatro no figuran en Wikidata.

Bibliografía

[1] *Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno*. Boletín Oficial del Estado, núm. 295, de 10 de diciembre de 2013 (BOE-A-2013-12887). https://www.boe.es/buscar/pdf/2013/BOE-A-2013-12887-consolidado.pdf

[2] OpenAI. *Introducing ChatGPT*. 30 de noviembre de 2022. https://openai.com/index/chatgpt/

[3] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., y Fung, P. *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys, vol. 55, núm. 12, art. 248, 2023. https://arxiv.org/pdf/2202.03629

[4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[5] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., y Wang, H. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997, 2023. https://arxiv.org/pdf/2312.10997

[6] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., y Larson, J. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024. https://arxiv.org/pdf/2404.16130

[7] Hogan, A., Blomqvist, E., Cochez, M., D'Amato, C., De Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., y Zimmermann, A. *Knowledge Graphs*. ACM Computing Surveys, vol. 54, núm. 4, art. 71, pp. 1–37, 2021. https://arxiv.org/pdf/2003.02320

[8] W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 de febrero de 2014. https://www.w3.org/TR/rdf11-concepts/

[9] Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F., y Rudolph, S. (eds.) *OWL 2 Web Ontology Language Primer (Second Edition)*. W3C Recommendation, 11 de diciembre de 2012. https://www.w3.org/TR/owl2-primer/

[10] W3C. *SPARQL 1.1 Query Language*. W3C Recommendation, 21 de marzo de 2013. https://www.w3.org/TR/sparql11-query/

[11] W3C. *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation, 18 de agosto de 2009. https://www.w3.org/TR/skos-reference/

[12] W3C. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 20 de julio de 2017. https://www.w3.org/TR/shacl/

[13] Nogueira, R., y Cho, K. *Passage Re-ranking with BERT*. arXiv:1901.04085, 2019. https://arxiv.org/pdf/1901.04085

[14] Cohere. *Rerank*. Consultado el 7 de octubre de 2026. https://cohere.com/rerank

[15] Giuliani, A., Manca, M. M., Piano, L., Podda, A. S., Pompianu, L., y Tiddia, S. G. *Are LLMs Adequate SPARQL Query Generators? Investigating Zero-Shot NL-to-SPARQL Translation*. Neural Computing and Applications, vol. 38, núm. 3, art. 33, 2026. https://doi.org/10.1007/s00521-025-11799-x

[16] Hernandez-Camero, I. V., Garcia-Lopez, E., Garcia-Cabot, A., y Caro-Alvaro, S. *Context-Aware Few-Shot Learning SPARQL Query Generation from Natural Language on an Aviation Knowledge Graph*. Machine Learning and Knowledge Extraction, vol. 7, núm. 2, art. 52, 2025. https://www.mdpi.com/2504-4990/7/2/52

[17] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., y Zhou, J. *Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation*. Proceedings of the VLDB Endowment (PVLDB), vol. 17, núm. 5, pp. 1132–1145, 2024. https://www.vldb.org/pvldb/vol17/p1132-gao.pdf

[18] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., y Chen, H. *CodeS: Towards Building Open-source Language Models for Text-to-SQL*. Proceedings of the ACM on Management of Data (SIGMOD 2024), vol. 2, núm. 3, pp. 1–28, 2024. https://arxiv.org/pdf/2402.16347

[19] Unger, C., Bühmann, L., Lehmann, J., Ngonga Ngomo, A.-C., Gerber, D., y Cimiano, P. *Template-based Question Answering over RDF Data*. En *Proceedings of the 21st International Conference on World Wide Web* (WWW 2012), pp. 639–648, 2012. https://dl.acm.org/doi/pdf/10.1145/2187836.2187923

[20] Vollmers, D., Jalota, R., Moussallem, D., Topiwala, H., Ngonga Ngomo, A.-C., y Usbeck, R. *Knowledge Graph Question Answering using Graph-Pattern Isomorphism*. En *Further with Knowledge Graphs: Proceedings of the 17th International Conference on Semantic Systems* (SEMANTiCS 2021), Studies on the Semantic Web, vol. 53, pp. 103–117, IOS Press, 2021. https://arxiv.org/pdf/2103.06752

[21] Arazzi, M., Ligari, D., Nicolazzo, S., y Nocera, A. *Augmented Knowledge Graph Querying leveraging LLMs*. En *2025 International Joint Conference on Neural Networks* (IJCNN 2025), 2025. https://arxiv.org/pdf/2502.01298

[22] Dirección General de Trabajo. *Resolución de 4 de abril de 2025 por la que se registra y publica el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública*. Boletín Oficial del Estado (BOE-A-2025-7766), 2025. https://www.boe.es/boe/dias/2025/04/16/pdfs/BOE-A-2025-7766.pdf

[23] Dirección General de Trabajo. *Resolución de 13 de abril de 2026 por la que se registra y publica el Acta de la Comisión paritaria por la que se aprueba la actualización de las tablas salariales del XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública*. Boletín Oficial del Estado (BOE-A-2026-9024), 2026. https://www.boe.es/boe/dias/2026/04/24/pdfs/BOE-A-2026-9024.pdf

[24] LangChain. *LangChain*. Consultado el 8 de octubre de 2026. https://www.langchain.com

[25] Ollama. *Ollama*. Consultado el 7 de octubre de 2026. https://ollama.com

[26] Groq. *Groq*. Consultado el 7 de octubre de 2026. https://groq.com

[27] Google. *Gemini API*. Consultado el 7 de octubre de 2026. https://ai.google.dev

[28] OpenAI. *gpt-oss-120b*. Consultado el 8 de octubre de 2026. https://huggingface.co/openai/gpt-oss-120b

[29] RDFLib. *rdflib*. Consultado el 7 de octubre de 2026. https://rdflib.readthedocs.io

[30] RDFLib. *OWL-RL*. Consultado el 7 de octubre de 2026. https://owl-rl.readthedocs.io

[31] Ontotext. *GraphDB*. Consultado el 7 de octubre de 2026. https://graphdb.ontotext.com

[32] Apache Software Foundation. *Apache Jena Fuseki*. Consultado el 7 de octubre de 2026. https://jena.apache.org/documentation/fuseki2/

[33] Blazegraph. *Blazegraph Database*. Consultado el 7 de octubre de 2026. https://blazegraph.com

[34] Chroma. *Chroma*. Consultado el 7 de octubre de 2026. https://www.trychroma.com

[35] Alibaba Cloud. *Qwen2.5*. Consultado el 8 de octubre de 2026. https://ollama.com/library/qwen2.5

[36] Alibaba Cloud. *Qwen3*. Consultado el 8 de octubre de 2026. https://ollama.com/library/qwen3

[37] Chainlit. *Chainlit*. Consultado el 8 de octubre de 2026. https://chainlit.io

[38] Bhat, S. R., Rudat, M., Spiekermann, J. y Flores-Herr, N. *Rethinking Chunk Size For Long-Document Retrieval: A Multi-Dataset Analysis*. arXiv:2505.21700, 2025. Consultado el 30 de octubre de 2026. https://arxiv.org/pdf/2505.21700

[39] Nomic AI. *nomic-embed-text*. Consultado el 8 de octubre de 2026. https://ollama.com/library/nomic-embed-text

[40] Es, S., James, J., Espinosa-Anke, L., y Schockaert, S. *RAGAs: Automated Evaluation of Retrieval Augmented Generation*. En *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (EACL 2024), pp. 150–158, 2024. https://aclanthology.org/2024.eacl-demo.16.pdf

[41] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., y Stoica, I. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. En *Advances in Neural Information Processing Systems 36* (NeurIPS 2023), Datasets and Benchmarks Track, 2023. https://arxiv.org/pdf/2306.05685

[42] Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., y Liu, Z. *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv:2402.03216, 2024. https://arxiv.org/pdf/2402.03216

[43] Holtzman, A., Buys, J., Du, L., Forbes, M., y Choi, Y. *The Curious Case of Neural Text Degeneration*. En *International Conference on Learning Representations* (ICLR 2020), 2020. https://arxiv.org/pdf/1904.09751

[44] Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. *A Survey on LLM-as-a-Judge*. arXiv:2411.15594, 2024. https://arxiv.org/pdf/2411.15594

[45] Sarmah, B., Hall, B., Rao, R., Patel, S., Pasquali, S., y Mehta, D. *HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction*. arXiv:2408.04948, 2024. https://arxiv.org/pdf/2408.04948

[46] Oficina de Publicaciones de la Unión Europea. *EuroVoc*. Consultado el 8 de octubre de 2026. https://op.europa.eu/en/web/eu-vocabularies/eurovoc

[47] Wikimedia Foundation. *Wikidata*. Consultado el 8 de octubre de 2026. https://www.wikidata.org

[48] Zarateman. *Bilbao - Ayuntamiento 4*, fotografía de la fachada del Ayuntamiento de Bilbao, publicada en Wikimedia Commons con licencia CC0. Consultado el 30 de octubre de 2026. https://commons.wikimedia.org/wiki/File:Bilbao_-_Ayuntamiento_4.JPG